

Introducción

El cáncer de colon es el tercer cáncer más frecuente a nivel mundial, y la segunda causa de muerte por cáncer, con aproximadamente 1.9 millones de casos nuevos reportados y más de 900,000 muertes registradas anualmente a nivel mundial, según las estimaciones más recientes de la Agencia Internacional para la Investigación del Cáncer (IARC). A pesar de los grandes avances en tratamientos, la tasa de supervivencia varía considerablemente entre pacientes con diagnósticos aparentemente similares, sugiriendo la existencia de subtipos moleculares con comportamientos biológicos distintos. El análisis de expresión génica mediante técnicas de aprendizaje no supervisado ofrece una oportunidad para descubrir estos subtipos de forma objetiva, sin depender de etiquetas clínicas previas.

En este proyecto, se estará analizando un dataset obtenido de la página web de la Universidad de California Santa Cruz. El dataset fue creado por el Genomic Data Commons, que es un repositorio de datos y plataforma computacional del Instituto Nacional de Cáncer de los Estados Unidos. Esta organización almacena, procesa y comparte datos genómicos y clínicos de cáncer para impulsar la medicina de precisión. El dataset a analizar es el conjunto GDC TCGA Colon Cancer (COAD); como su nombre indica, es un dataset sobre cancer de colon que cuenta con tres archivos diferentes relevantes para este proyecto. El primer archivo es "STAR-TPM" (expresión génica), y contiene datos moleculares obtenidos secuenciando el ARN (Ácido Ribonucleico) del tumor, es decir el conjunto de moléculas ARN presentes en las células cancerosas. El análisis de este conjunto es de suma importancia para los médicos, pues les permite observar que están haciendo las células del tumor en tiempo real. En este conjunto se observan los niveles de actividad de 60,660 genes en cada paciente. El segundo archivo a analizar es "Clinical", este dataset contiene la información médica de cada paciente: su edad, la fase en la que se encuentra el tumor, el tipo de muestra... Son los datos clínicos del paciente, que provienen de su expediente médico. El último archivo que se estará utilizando es "Survival". En este archivo se encuentra el seguimiento del paciente, es decir si el paciente sobrevivió y cuantos días o si falleció y son datos registrados con el tiempo después del diagnóstico. La razón del porque analizamos 3 archivos distintos es porque los 3 nos muestran información valiosa para el modelo a realizar y entender la evolución del paciente. Estos 3 archivos están separados debido a que cada uno es generado de manera distinta:

- La expresión génica viene del laboratorio de secuenciación.
- Los datos médicos son capturados por el médico.
- Los datos de "Survival" se registran en el seguimiento a largo plazo.

Estos archivos también serán utilizados de manera distinta y en diferentes etapas a lo largo de este proyecto. El objetivo de este proyecto es aplicar técnicas de reducción de

dimensionalidad y clustering sobre los datos de expresión génica del dataset TCGA-COAD, con el fin de identificar agrupaciones naturales de pacientes con perfiles moleculares similares. Se espera que los grupos obtenidos reflejen patrones moleculares reales del cáncer de colon, y que cada grupo presente características clínicas distintas en variables como el estadio del tumor, la edad del paciente al momento del diagnóstico y su supervivencia. Es importante señalar que el proceso de clustering se realizará exclusivamente con los datos de expresión génica; los datos clínicos y de supervivencia serán incorporados únicamente en la etapa de interpretación de resultados.

Exploración del conjunto de datos

Para empezar a realizar la exploración, primero cargamos los cuatro archivos del dataset TCGA-COAD: la matriz de expresión génica, el mapeo de IDs de genes, los datos clínicos y los datos de supervivencia. Así, podemos observar el número total de genes, muestras y variables presentes en los diferentes datasets.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Configuración visual
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['font.size'] = 12
sns.set_style("whitegrid")

# Carga de archivos
expr = pd.read_csv('TCGA-COAD.star_tpm.tsv', sep='\t', index_col=0)
gene_map = pd.read_csv('gencode.v36.annotation.gtf.gene.probemaps', sep='\t')
clin = pd.read_csv('TCGA-COAD.clinical.tsv', sep='\t')
surv = pd.read_csv('TCGA-COAD.survival.tsv', sep='\t')

print("Archivos cargados correctamente.")
print(f" Expresión génica : {expr.shape[0]:,} genes x {expr.shape[1]} mue")
print(f" Gene mapping      : {gene_map.shape[0]:,} genes")
print(f" Clinical           : {clin.shape[0]} muestras x {clin.shape[1]} va")
print(f" Survival            : {surv.shape[0]} muestras")
```

```
Archivos cargados correctamente.
Expresión génica : 60,660 genes x 514 muestras
Gene mapping      : 60,660 genes
Clinical           : 562 muestras x 86 variables
Survival            : 530 muestras
```

Matriz de Expresión Génica

Posteriormente, realizamos la Matriz de Expresión Génica, que contiene un total de 60,660 genes como filas y 514 pacientes como columnas. Los valores representan niveles de expresión en escala $\log_2(\text{TPM}+1)$, que es una transformación estándar para estabilizar la varianza. No se encontraron valores nulos, es decir, todos los 514 pacientes tienen un valor registrado para los 60,660 genes, ninguna celda de la matriz se encuentra vacía. Se muestra un rango de valores amplio de 0 a 17.873 en la escala $\log_2(\text{TPM}+1)$, los valores indican el nivel de ARN que estaba produciendo cada gen, cuanto mayor es el número, más activo se encontraba el gen en algún paciente. La media global es de 1.1110, indica el promedio de todos los valores presentes en la matriz, esto indica que la mayoría de genes tienen poca actividad, y hay unos pocos muy activos que tienen un gran impacto en el tumor.

```
In [2]: print("=== MATRIZ DE EXPRESIÓN GÉNICA ===")
print(f"\nDimensiones : {expr.shape[0]:,} genes × {expr.shape[1]} pacientes")
print(f"Valores nulos: {expr.isnull().sum().sum()}")
print(f"\nRango de valores : {expr.values.min():.3f} - {expr.values.max():.3f}")
print(f"Media global      : {expr.values.mean():.4f}")
print(f"\nPrimeras 5 filas:")
print(expr.iloc[:5, :4])
```

```
=== MATRIZ DE EXPRESIÓN GÉNICA ===
```

```
Dimensiones : 60,660 genes × 514 pacientes
Valores nulos: 0
```

```
Rango de valores : 0.000 - 17.873
Media global      : 1.1110
```

```
Primeras 5 filas:
```

	TCGA-5M-AAT6-01A	TCGA-AA-3552-01A	TCGA-AA-3867-01A	\
Ensembl_ID				
ENSG00000000003.15	4.783671	7.069015	7.627462	
ENSG00000000005.6	0.343692	0.753776	0.300417	
ENSG000000000419.13	6.679223	6.679985	7.430763	
ENSG000000000457.14	2.640366	2.605779	2.751292	
ENSG000000000460.17	2.678906	1.893168	2.264747	

	TCGA-AD-6895-01A
Ensembl_ID	
ENSG00000000003.15	4.079737
ENSG00000000005.6	0.000000
ENSG000000000419.13	6.839957
ENSG000000000457.14	3.088125
ENSG000000000460.17	2.415056

Distribución de Expresión Génica

Para entender mejor el comportamiento general de los datos, se analizó la distribución de los valores de expresión a través de dos gráficas:

- **Distribución Global de Expresión Génica** : Muestra que la gran mayoría de los valores se concentran entre 0 y $2 \log_2(\text{TPM}+1)$, con una media de 1.12. La distribución tiene forma de J invertida, muchos genes con expresión muy baja o cero, y pocos genes con expresión muy alta. Esto es completamente normal en datos de RNAseq, ya que en cualquier tipo de célula solo una fracción de los genes está activa simultáneamente.
- **Porcentaje de Pacientes con Expresión = 0 por Gen**: Muestra un patrón muy claro pues hay un gran grupo de genes con expresión cero en casi ningún paciente (genes activos), y otro gran grupo con expresión cero en casi el 100% de los pacientes (genes inactivos). Específicamente:
 - 31,960 genes tienen expresión cero en más del 50% de los pacientes
 - 19,197 genes tienen expresión cero en más del 90% de los pacientes
 - Solo 12,846 genes tienen expresión mayor a cero en todos los pacientes

Esto justifica directamente el filtrado de genes de baja varianza que se realizará en la Sección 3 donde la mayoría de los genes están prácticamente apagados en casi todos los pacientes y no aportarán información útil para el clustering.

```
In [3]: # Distribución de valores de expresión génica
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

sample_vals = expr.values.flatten()
np.random.seed(42)
sample_vals = np.random.choice(sample_vals, size=10000, replace=False)

axes[0].hist(sample_vals, bins=50, color='steelblue', edgecolor='white')
axes[0].set_title('Distribución Global de Expresión Génica')
axes[0].set_xlabel('log2(TPM+1)')
axes[0].set_ylabel('Frecuencia')
axes[0].axvline(np.mean(sample_vals), color='red', linestyle='--',
                label=f'Media: {np.mean(sample_vals):.2f}')
axes[0].legend()

zeros_per_gene = (expr == 0).sum(axis=1)
pct_zeros = (zeros_per_gene / expr.shape[1]) * 100

axes[1].hist(pct_zeros, bins=40, color='coral', edgecolor='white')
axes[1].set_title('% de Pacientes con Expresión = 0 por Gen')
axes[1].set_xlabel('% de muestras con valor 0')
axes[1].set_ylabel('Número de genes')

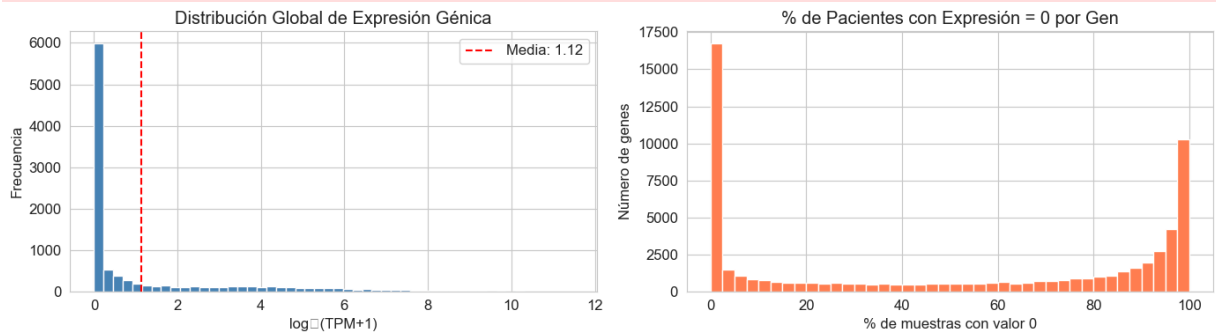
plt.tight_layout()
plt.savefig('fig_distribucion_expresion.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"Genes con expresión 0 en más del 50% de pacientes: {(pct_zeros > 50)}")
print(f"Genes con expresión 0 en más del 90% de pacientes: {(pct_zeros > 90)}")
print(f"Genes con expresión > 0 en todos los pacientes : {(pct_zeros == 0)}")
```

```

/var/folders/px/dlp0zrrn4w16nb0rn551ywd80000gn/T/ipykernel_1236/1873204102.p
y:24: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Aria
l.
plt.tight_layout()
/var/folders/px/dlp0zrrn4w16nb0rn551ywd80000gn/T/ipykernel_1236/1873204102.p
y:25: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Aria
l.
plt.savefig('fig_distribucion_expresion.png', dpi=150, bbox_inches='tigh
t')
/Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packa
ges/IPython/core/pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TW
O}) missing from font(s) Arial.
fig.canvas.print_figure(bytes_io, **kw)

```



Genes con expresión 0 en más del 50% de pacientes: 31,960

Genes con expresión 0 en más del 90% de pacientes: 19,197

Genes con expresión > 0 en todos los pacientes : 12,846

Variables clínicas clave

El archivo clínico contiene un total de 86 variables por paciente, por lo que se incluye un diccionario para las variables más relevantes para el análisis, así como el número de valores faltantes en cada una. Las variables clave seleccionadas fueron:

- **gender.demographic:** Indica el sexo del paciente y posee 2 valores faltantes.
- **age_at_index.demographic:** Indica la edad al momento del diagnóstico (en años) y posee 2 valores faltantes.
- **vital_status.demographic:** Indica el estado vital (Vivo / Fallecido) y posee 2 valores faltantes.
- **race.demographic:** Indica la raza/etnia del paciente y posee 2 valores faltantes.
- **ajcc_pathologic_stage.diagnoses:** Indica el estadio clínico del tumor según la clasificación AJCC (Stage I, II, III, IV) y posee 14 valores faltantes. Es la variable clínica más importante para este análisis.
- **ajcc_pathologic_t.diagnoses:** Describe el tamaño e invasión local del tumor (T) y posee 2 valores faltantes.
- **ajcc_pathologic_n.diagnoses:** Indica si el cáncer se ha extendido a los ganglios linfáticos cercanos (N) y posee 2 valores faltantes.
- **ajcc_pathologic_m.diagnoses:** Indica si existe metástasis a órganos distantes (M) y posee 10 valores faltantes.

- **sample_type.samples:** Indica el tipo de muestra recolectada (tumor primario, tejido normal, metastásico) y no posee valores faltantes.

Las variables restantes fueron descartadas en diversas categorías, por ejemplo por ser identificadores internos que no aportan información clínica como lo son **sample**, **id**, **case_id**... variables con un solo valor único debido a que esa variable no podrá diferenciar nada entre clusters, o por presentar un número excesivo de valores faltantes (70%) que las hace poco útiles para el análisis porque la información que se obtendría sería sobre una minoría de pacientes y no sería representativo, como **year_of_death.demographic** que cuenta con un 88% de valores faltantes.

Adicionalmente, se eliminaron variables que duplicaban información ya capturada por otras, como **days_to_birth.demographic** y **year_of_birth.demographic**, que contienen la misma información que **age_at_index.demographic** expresada de forma distinta. Otras categorías que también se tuvieron en cuenta para realizar los descartes fueron las variables de logística de tratamiento y muestra que expresaban como se procesó la muestra en el laboratorio o metadatos administrativos del proyecto que tienen 1 solo valor porque todas pertenecen al mismo proyecto.

```
In [4]: # Variables disponibles en el archivo clínico
print(f"Total de variables clínicas: {clin.shape[1]}\n")
print("Lista completa de variables:")
for i, col in enumerate(clin.columns, 1):
    n_nulos = clin[col].isnull().sum()
    n_unicos = clin[col].nunique()
    print(f" {i:>3}. {col:<50} | nulos: {n_nulos:>3} | valores únicos: {n_u
```

Total de variables clínicas: 86

Lista completa de variables:

1. sample	nulos: 0 val
ores únicos: 562	
2. id	nulos: 0 val
ores únicos: 461	
3. disease_type	nulos: 0 val
ores únicos: 4	
4. case_id	nulos: 0 val
ores únicos: 461	
5. submitter_id	nulos: 0 val
ores únicos: 461	
6. primary_site	nulos: 0 val
ores únicos: 2	
7. alcohol_history.exposures	nulos: 2 val
ores únicos: 1	
8. race.demographic	nulos: 2 val
ores únicos: 5	
9. gender.demographic	nulos: 2 val
ores únicos: 2	
10. ethnicity.demographic	nulos: 2 val
ores únicos: 3	
11. vital_status.demographic	nulos: 2 val
ores únicos: 2	
12. age_at_index.demographic	nulos: 2 val
ores únicos: 58	
13. days_to_birth.demographic	nulos: 4 val
ores únicos: 430	
14. year_of_birth.demographic	nulos: 2 val
ores únicos: 64	
15. year_of_death.demographic	nulos: 496 val
ores únicos: 14	
16. primary_site.project	nulos: 0 val
ores únicos: 1	
17. project_id.project	nulos: 0 val
ores únicos: 1	
18. disease_type.project	nulos: 0 val
ores únicos: 1	
19. name.project	nulos: 0 val
ores únicos: 1	
20. name.program.project	nulos: 0 val
ores únicos: 1	
21. tissue_source_site_id.tissue_source_site	nulos: 0 val
ores únicos: 25	
22. code.tissue_source_site	nulos: 0 val
ores únicos: 25	
23. name.tissue_source_site	nulos: 0 val
ores únicos: 25	
24. project.tissue_source_site	nulos: 0 val
ores únicos: 1	
25. bcr_id.tissue_source_site	nulos: 0 val
ores únicos: 2	
26. days_to_death.demographic	nulos: 438 val
ores únicos: 85	
27. entity_submitter_id.annotations	nulos: 474 val

ores únicos: 69	
28. notes.annotations	nulos: 474 val
ores únicos: 61	
29. submitter_id.annotations	nulos: 474 val
ores únicos: 69	
30. classification.annotations	nulos: 474 val
ores únicos: 7	
31. entity_id.annotations	nulos: 474 val
ores únicos: 69	
32. created_datetime.annotations	nulos: 474 val
ores únicos: 20	
33. annotation_id.annotations	nulos: 474 val
ores únicos: 69	
34. entity_type.annotations	nulos: 474 val
ores únicos: 3	
35. updated_datetime.annotations	nulos: 474 val
ores únicos: 7	
36. case_id.annotations	nulos: 474 val
ores únicos: 69	
37. state.annotations	nulos: 474 val
ores únicos: 3	
38. category.annotations	nulos: 474 val
ores únicos: 18	
39. status.annotations	nulos: 474 val
ores únicos: 3	
40. case_submitter_id.annotations	nulos: 474 val
ores únicos: 69	
41. synchronous_malignancy.diagnoses	nulos: 2 val
ores únicos: 3	
42. ajcc_pathologic_stage.diagnoses	nulos: 14 val
ores únicos: 13	
43. days_to_diagnosis.diagnoses	nulos: 2 val
ores únicos: 1	
44. last_known_disease_status.diagnoses	nulos: 2 val
ores únicos: 1	
45. tissue_or_organ_of_origin.diagnoses	nulos: 2 val
ores únicos: 9	
46. days_to_last_follow_up.diagnoses	nulos: 59 val
ores únicos: 277	
47. age_at_diagnosis.diagnoses	nulos: 4 val
ores únicos: 430	
48. primary_diagnosis.diagnoses	nulos: 2 val
ores únicos: 7	
49. prior_malignancy.diagnoses	nulos: 2 val
ores únicos: 2	
50. year_of_diagnosis.diagnoses	nulos: 2 val
ores únicos: 16	
51. prior_treatment.diagnoses	nulos: 2 val
ores únicos: 2	
52. ajcc_staging_system_edition.diagnoses	nulos: 69 val
ores únicos: 4	
53. ajcc_pathologic_t.diagnoses	nulos: 2 val
ores únicos: 7	
54. morphology.diagnoses	nulos: 2 val
ores únicos: 7	
55. ajcc_pathologic_n.diagnoses	nulos: 2 val


```

ores únicos: 8
  56. ajcc_pathologic_m.diagnoses | nulos: 10 | val
ores únicos: 5
  57. classification_of_tumor.diagnoses | nulos: 2 | val
ores únicos: 1
  58. icd_10_code.diagnoses | nulos: 2 | val
ores únicos: 9
  59. site_of_resection_or_biopsy.diagnoses | nulos: 2 | val
ores únicos: 9
  60. tumor_grade.diagnoses | nulos: 2 | val
ores únicos: 1
  61. progression_or_recurrence.diagnoses | nulos: 2 | val
ores únicos: 1
  62. age_at_earliest_diagnosis.diagnoses.xena_derived | nulos: 4 | val
ores únicos: 430
  63. age_at_earliest_diagnosis_in_years.diagnoses.xena_derived | nulos:
4 | valores únicos: 430
  64. updated_datetime.treatments.diagnoses | nulos: 2 | val
ores únicos: 459
  65. treatment_id.treatments.diagnoses | nulos: 2 | val
ores únicos: 459
  66. submitter_id.treatments.diagnoses | nulos: 2 | val
ores únicos: 459
  67. treatment_type.treatments.diagnoses | nulos: 2 | val
ores únicos: 2
  68. state.treatments.diagnoses | nulos: 2 | val
ores únicos: 1
  69. treatment_or_therapy.treatments.diagnoses | nulos: 2 | val
ores únicos: 9
  70. created_datetime.treatments.diagnoses | nulos: 2 | val
ores únicos: 459
  71. sample_type_id.samples | nulos: 0 | val
ores únicos: 4
  72. tumor_descriptor.samples | nulos: 0 | val
ores únicos: 4
  73. sample_id.samples | nulos: 0 | val
ores únicos: 562
  74. sample_type.samples | nulos: 0 | val
ores únicos: 4
  75. specimen_type.samples | nulos: 0 | val
ores únicos: 2
  76. composition.samples | nulos: 2 | val
ores únicos: 1
  77. is_ffpe.samples | nulos: 0 | val
ores únicos: 2
  78. tissue_type.samples | nulos: 0 | val
ores únicos: 2
  79. preservation_method.samples | nulos: 0 | val
ores únicos: 3
  80. days_to_collection.samples | nulos: 433 | val
ores únicos: 104
  81. initial_weight.samples | nulos: 441 | val
ores únicos: 57
  82. intermediate_dimension.samples | nulos: 135 | val
ores únicos: 21
  83. pathology_report_uuid.samples | nulos: 85 | val

```

```

ores únicos: 464
  84. shortest_dimension.samples | nulos: 135 | val
ores únicos: 10
  85. oct_embedded.samples      | nulos: 431 | val
ores únicos: 2
  86. longest_dimension.samples | nulos: 135 | val
ores únicos: 26

```

Distribución de Variables Clínicas

Posteriormente, creamos y observamos diversas gráficas que permitan entender como se distribuyen los valores de las diferentes variables clínicas seleccionadas.

- **Estadio Tumoral:** Es un sistema que utilizan los médicos para describir la extensión y propagación del cáncer en el cuerpo de una persona, una forma de medir qué tanto ha avanzado la enfermedad desde su punto de origen. La mayoría de los pacientes se encuentran en Stage IIA (159 pacientes), seguido de Stage I (88) y Stage IIIB (69). Esto indica que el dataset tiene mayor representación de estadios tempranos e intermedios, con menos pacientes en estadios avanzados como Stage IV (58) y Stage IVA (18).
- **Sexo:** La distribución es prácticamente equitativa, con un 52% hombres y 48% mujeres, lo que garantiza que el análisis no estará sesgado por género.
- **Edad:** La distribución de edad es aproximadamente normal con sesgo hacia la derecha. La mediana es de 69 años, con pacientes entre 31 y 90 años, lo cual es consistente con la literatura que indica que el cáncer de colon es más frecuente en adultos mayores.
- **Estado vital:** El 78% de los pacientes está vivo (436) al momento del registro, mientras que el 22% (124) falleció. Esta proporción permitirá analizar si los clusters identificados tienen diferencias significativas en supervivencia.
- **Tipo de muestra:** Se refiere a la naturaleza biológica y procedencia del tejido que se extrajo del paciente para ser analizado en el estudio. La mayoría corresponde a tumor primario (tejido tomado del lugar original donde el cáncer comenzó a crecer) con 474 muestras, seguido de tejido normal sano (tejido del mismo órgano pero que no presenta enfermedad) con 85, con 2 muestras metastásicas (tejido de un tumor que se propagó a un órgano distinto) y 1 recurrente (tejido de un tumor que volvió a aparecer en el mismo lugar después de haber sido tratado o eliminado). Para el análisis de clustering se utilizarán únicamente las muestras de tumor primario, eliminando las demás para evitar contaminar los patrones moleculares.
- **Tiempo de seguimiento:** El seguimiento varía entre 1 y 4,502 días, con una mediana de 731 días (aproximadamente 2 años). La distribución está fuertemente sesgada hacia la derecha, lo que indica que la mayoría de los pacientes tiene seguimientos relativamente cortos, con algunos casos de seguimiento muy prolongado.

```

In [5]: fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Distribución de Variables Clínicas – TCGA-COAD', fontsize=14,

# 1. Estadío tumoral
stage_counts = clin['ajcc_pathologic_stage.diagnoses'].value_counts()
axes[0,0].bar(stage_counts.index, stage_counts.values, color='steelblue', ec
axes[0,0].set_title('Estadio Tumoral (AJCC)')
axes[0,0].set_xlabel('Estadio')
axes[0,0].set_ylabel('Número de pacientes')
axes[0,0].tick_params(axis='x', rotation=45)

# 2. Sexo
gender_counts = clin['gender.demographic'].value_counts()
axes[0,1].pie(gender_counts.values, labels=['Hombre', 'Mujer'],
              autopct='%1.1f%%', colors=['#4e79a7', '#f28e2b'], startangle=9
axes[0,1].set_title('Distribución por Sexo')

# 3. Edad
axes[0,2].hist(clin['age_at_index.demographic'].dropna(), bins=20,
               color='teal', edgecolor='white')
axes[0,2].set_title('Distribución de Edad al Diagnóstico')
axes[0,2].set_xlabel('Edad (años)')
axes[0,2].set_ylabel('Número de pacientes')
axes[0,2].axvline(clin['age_at_index.demographic'].median(), color='red',
                  linestyle='--', label=f'Mediana: {clin["age_at_index.demog
axes[0,2].legend()

# 4. Estado vital
vital_counts = clin['vital_status.demographic'].value_counts()
axes[1,0].bar(vital_counts.index, vital_counts.values,
              color=['#59a14f', '#e15759'], edgecolor='white')
axes[1,0].set_title('Estado Vital')
axes[1,0].set_ylabel('Número de pacientes')

# 5. Tipo de muestra
sample_counts = clin['sample_type.samples'].value_counts()
axes[1,1].bar(sample_counts.index, sample_counts.values, color='slateblue',
axes[1,1].set_title('Tipo de Muestra')
axes[1,1].tick_params(axis='x', rotation=30)
axes[1,1].set_ylabel('Número de pacientes')

# 6. Supervivencia (días)
axes[1,2].hist(surv['OS.time'].dropna(), bins=30, color='coral', edgecolor='
axes[1,2].set_title('Tiempo de Seguimiento (Survival)')
axes[1,2].set_xlabel('Días')
axes[1,2].set_ylabel('Número de pacientes')
axes[1,2].axvline(surv['OS.time'].median(), color='darkred',
                  linestyle='--', label=f'Mediana: {surv["OS.time"].median()
axes[1,2].legend()

plt.tight_layout()
plt.savefig('fig_distribucion_clinica.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figura guardada.")

```

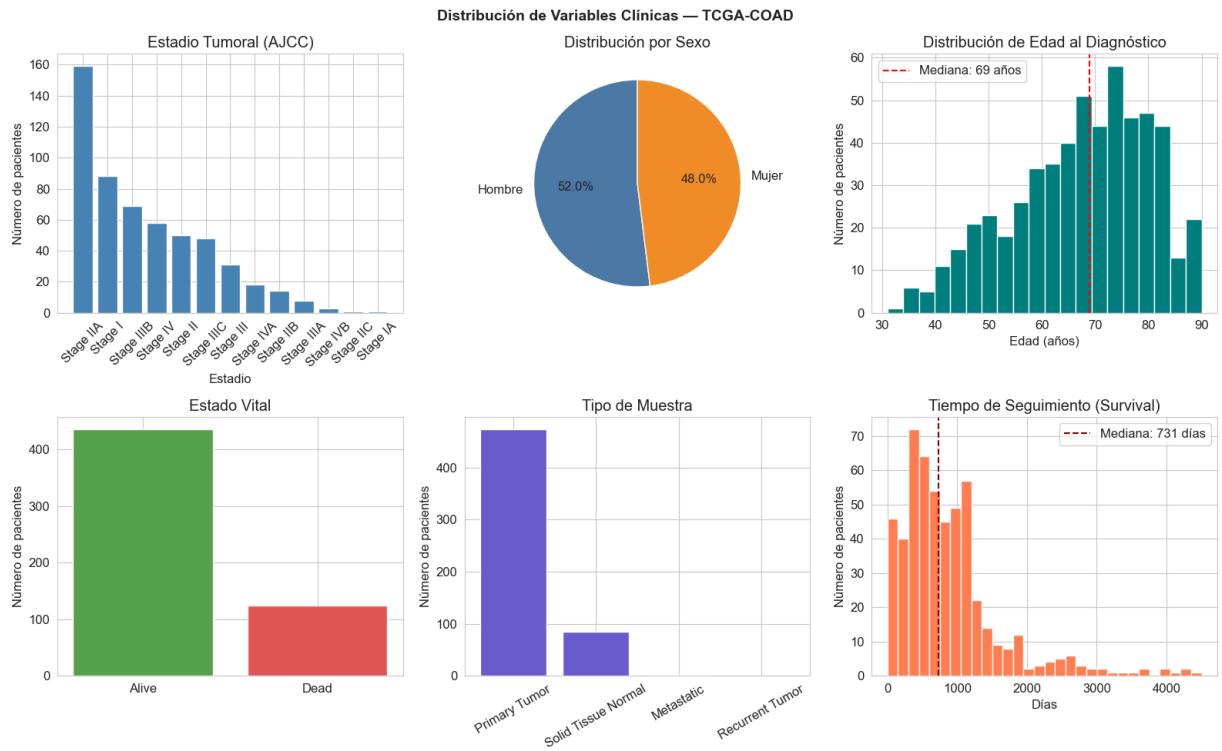


Figura guardada.

Valores faltantes

Se realizó una revisión exhaustiva de valores faltantes en los tres archivos. La matriz de expresión génica no presenta ningún valor nulo, lo que significa que todos los 514 pacientes tienen un valor registrado para los 60,660 genes — ninguna celda está vacía. Esto simplifica considerablemente la preparación de datos pues no se requiere imputación en el archivo principal de análisis. En los datos clínicos, las variables con mayor número de faltantes son el estadio tumoral (14 valores) y la variable de metástasis M (10 valores). Esto es esperado ya que no todos los pacientes cuentan con estadificación completa en su expediente. Estas variables se seguirán utilizando en la interpretación clínica, tomando en cuenta que un pequeño porcentaje de pacientes no tendrá dato disponible. Los datos de supervivencia están completamente limpios, con 0 valores nulos tanto en tiempo de seguimiento como en el evento de fallecimiento, lo que los hace directamente utilizables sin ningún tratamiento adicional. Esta celda se genera para cumplir con el requisito metodológico de documentar la calidad de los datos antes de proceder al análisis, garantizando transparencia en las decisiones tomadas posteriormente.

```
In [7]: print("=== VALORES FALTANTES ===\n")
print("Expresión génica:", expr.isnull().sum().sum(), "valores nulos ✅")
print()
key_cols = {
    'gender.demographic': 'Sexo del paciente',
    'age_at_index.demographic': 'Edad al diagnóstico (años)',
    'vital_status.demographic': 'Estado vital (Vivo / Fallecido)',
    'race.demographic': 'Raza / etnia',
```

```

    'ajcc_pathologic_stage.diagnoses': 'Estadio clínico del tumor (I-IV)',
    'ajcc_pathologic_t.diagnoses': 'Tamaño del tumor (T)',
    'ajcc_pathologic_n.diagnoses': 'Afectación ganglionar (N)',
    'ajcc_pathologic_m.diagnoses': 'Metástasis (M)',
    'sample_type.samples': 'Tipo de muestra (tumor / normal)'
}

missing_clin = clin[list(key_cols.keys())].isnull().sum()
missing_clin = missing_clin[missing_clin > 0]
print("Variables clínicas con valores faltantes:")
print(missing_clin.to_string())
print()
print(f"Survival - OS.time: {surv['OS.time'].isnull().sum()} nulos")
print(f"Survival - OS      : {surv['OS'].isnull().sum()} nulos")

```

=== VALORES FALTANTES ===

Expresión génica: 0 valores nulos 

Variables clínicas con valores faltantes:

gender.demographic	2
age_at_index.demographic	2
vital_status.demographic	2
race.demographic	2
ajcc_pathologic_stage.diagnoses	14
ajcc_pathologic_t.diagnoses	2
ajcc_pathologic_n.diagnoses	2
ajcc_pathologic_m.diagnoses	10

Survival - OS.time: 0 nulos

Survival - OS : 0 nulos

Búsqueda y detección de Outliers

Para identificar muestras atípicas se calculó la expresión media de cada paciente sobre todos sus genes, y se aplicó un z-score para detectar aquellas con valores inusualmente altos o bajos respecto al resto de la cohorte. La búsqueda de outliers se realiza únicamente en la matriz de expresión génica y no en los datos clínicos, porque es precisamente la expresión génica la que se usará para construir los clusters. Un paciente con un perfil de expresión anormalmente distinto al resto podría formar un cluster propio simplemente por razones técnicas y no por razones biológicas reales. Detectarlos antes del clustering evita que contaminen los resultados. Los datos clínicos, al no participar en el clustering, no requieren este tipo de revisión en esta etapa. Para la detección de los Outliers se utilizó el z-score, un método equivalente al criterio de Tukey en cuanto a que ambos identifican valores que se alejan significativamente de la distribución central. Se aplicó un umbral de $z > 3$, es decir, muestras cuya expresión media se encuentra a más de 3 desviaciones estándar de la media del grupo.

- **Histograma:** Muestra que la gran mayoría de muestras tiene una expresión media concentrada entre 1.0 y 1.3 $\log_2(\text{TPM}+1)$, formando una distribución

aproximadamente normal con ligero sesgo a la derecha. Esto indica homogeneidad general entre los pacientes.

- **Boxplot:** Confirma visualmente las 8 muestras detectadas como puntos aislados por encima del bigote superior, todas con expresión media entre 1.51 y 2.04.

Las 8 muestras outliers corresponden todas a códigos TCGA-A6, lo que sugiere que podrían provenir de un sitio de recolección específico con características técnicas distintas. Sin embargo, dado que sus valores no son biológicamente imposibles, se decide mantenerlas en el análisis y monitorear su comportamiento en etapas posteriores. Si forman un cluster propio, se documentará como grupo técnico y se interpretará con cautela, sin que ello afecte las conclusiones sobre los subtipos moleculares principales.

```
In [10]: sample_means = expr.mean(axis=0)
sample_stds = expr.std(axis=0)

# Z-score de las medias
z_scores = np.abs(stats.zscore(sample_means))
outliers = sample_means[z_scores > 3]

print(f"=== OUTLIERS EN EXPRESIÓN GÉNICA ===\n")
print(f"Expresión media por muestra - Rango: {sample_means.min():.3f} - {sample_means.max():.3f}")
print(f"Muestras con z-score > 3 (posibles outliers): {len(outliers)}")
if len(outliers) > 0:
    print(outliers)

# Boxplot de expresión media por muestra
fig, axes = plt.subplots(1, 2, figsize=(14, 4))

axes[0].hist(sample_means, bins=40, color='steelblue', edgecolor='white')
axes[0].set_title('Expresión Media por Muestra')
axes[0].set_xlabel('Media log2(TPM+1)')
axes[0].set_ylabel('Número de muestras')

axes[1].boxplot(sample_means, vert=True, patch_artist=True,
                boxprops=dict(facecolor='lightblue'))
axes[1].set_title('Boxplot - Expresión Media por Muestra')
axes[1].set_ylabel('Media log2(TPM+1)')

plt.tight_layout()
plt.savefig('fig_outliers_expresion.png', dpi=150, bbox_inches='tight')
plt.show()
print("Figura guardada.")
```

=== OUTLIERS EN EXPRESIÓN GÉNICA ===

Expresión media por muestra – Rango: 0.773 – 2.045

Muestras con z-score > 3 (posibles outliers): 8

TCGA-A6-6781-01B	1.574525
TCGA-A6-2677-01B	1.868061
TCGA-CM-4748-01A	1.513204
TCGA-A6-2684-01C	1.665546
TCGA-A6-6650-01B	1.680552
TCGA-A6-3810-01B	1.909057
TCGA-A6-2674-01B	2.018412
TCGA-A6-5656-01B	2.044726

dtype: float64

```
/var/folders/px/dlp0zrrn4w16nb0rn551ywd80000gn/T/ipykernel_1236/1963677946.p
y:27: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Aria
l.
```

```
plt.tight_layout()
```

```
/var/folders/px/dlp0zrrn4w16nb0rn551ywd80000gn/T/ipykernel_1236/1963677946.p
y:28: UserWarning: Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Aria
l.
```

```
plt.savefig('fig_outliers_expression.png', dpi=150, bbox_inches='tight')
/Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packa
ges/IPython/core/pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT TW
O}) missing from font(s) Arial.
```

```
fig.canvas.print_figure(bytes_io, **kw)
```

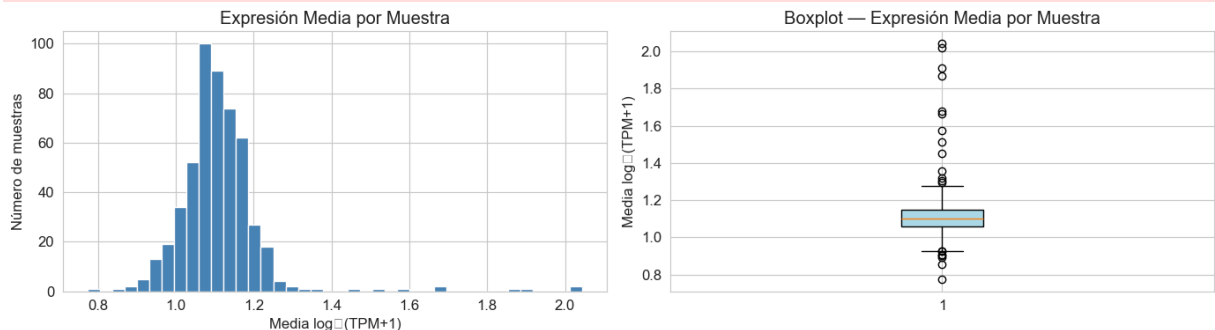


Figura guardada.

Preparación de datos

La preparación de los datos se realizó en cuatro pasos: filtrado de muestras, mapeo de genes, filtrado por varianza y estandarización. Cada decisión tomada se justifica a continuación. Primero, empezamos filtrando las 514 muestras originales para conservar únicamente las 471 de tumor primario, eliminando 43 que correspondían a tejido normal sano, muestras metastásicas y recurrentes. Esto es fundamental porque mezclar tejido normal con tumoral introduciría diferencias biológicas artificiales que dominarían el clustering, ocultando los subtipos reales del cáncer.

```
In [11]: # PASO 1 – Filtrar solo muestras de tumor primario
tumor_samples = clin[clin['sample_type.samples'] == 'Primary Tumor']['sample
expr_tumor = expr[[s for s in expr.columns if s in tumor_samples]]

print(f"Muestras originales      : {expr.shape[1]}")
```

```
print(f"Muestras tumor primario : {expr_tumor.shape[1]}")
print(f"Muestras eliminadas      : {expr.shape[1] - expr_tumor.shape[1]}")
```

```
Muestras originales      : 514
Muestras tumor primario  : 471
Muestras eliminadas      : 43
```

Mapeo de IDs de Genes

En la matriz de expresión génica, los genes están identificados con códigos Ensembl como ENSG00000000003, que son identificadores únicos y precisos pero completamente ininterpretables para un análisis biológico. Para poder interpretar los resultados del clustering en términos biológicos reales, es necesario convertirlos a nombres reconocibles como TSPAN6, KRAS o TP53.

Existen dos sistemas de nomenclatura génica con propósitos distintos. El Ensembl ID funciona como un número de serie único que nunca cambia, garantizando que cada gen sea identificado sin ambigüedad independientemente de cómo haya sido llamado históricamente. El símbolo HGNC (Human Genome Nomenclature Committee) es el nombre oficial aprobado por el organismo internacional que estandariza los nombres de todos los genes humanos: es el nombre que aparece en artículos científicos, expedientes médicos y bases de datos clínicas.

Este paso es especialmente crítico para las secciones posteriores del proyecto, donde se deberá identificar qué genes diferencian cada cluster. Sin este mapeo, sería imposible conectar los resultados con la literatura científica. Por ejemplo, encontrar que el cluster 1 tiene alta expresión de ENSG00000133703 no dice nada, pero saber que ese código corresponde a KRAS sí permite una interpretación clínica significativa mediante la búsqueda del código en el HGNC.

Para el mapeo se utilizó el archivo de referencia gencode v36, que es el estándar utilizado por TCGA para sus datos. Se eliminó primero el sufijo de versión de cada ID (ej. ENSG00000000003.15 → ENSG00000000003) para garantizar compatibilidad. Los genes duplicados fueron eliminados conservando el primero. Tras el mapeo quedaron 59,427 genes con nombres HGNC reconocibles listos para el siguiente paso. La eliminación de 1,233 genes duplicados representa apenas el 2% del total y no compromete el análisis, ya que los genes biológicamente relevantes para el cáncer de colon son únicos en el genoma, que es el conjunto completo de toda la información genética de un organismo.

```
In [12]: # PASO 2 – Mapear Ensembl IDs a nombres de genes
gene_map_clean = gene_map.copy()
gene_map_clean['id_clean'] = gene_map_clean['id'].str.split('.').str[0]

# Limpiar versión del ID en la matriz
expr_tumor.index = expr_tumor.index.str.split('.').str[0]

# Crear diccionario de mapeo
```



```

id_to_gene = dict(zip(gene_map_clean['id_clean'], gene_map_clean['gene']))

# Mapear – si no encuentra el ID, conserva el ID original
new_index = [id_to_gene.get(g, g) for g in expr_tumor.index]
expr_tumor.index = new_index

# Eliminar genes duplicados
expr_tumor = expr_tumor[~expr_tumor.index.duplicated(keep='first')]

print(f"Genes después del mapeo : {expr_tumor.shape[0]:,}")
print(f"Primeros 5 genes          : {list(expr_tumor.index[:5])}")

```

Genes después del mapeo : 59,427

Primeros 5 genes : ['TSPAN6', 'TNMD', 'DPM1', 'SCYL3', 'C1orf112']

Filtrado de genes de baja varianza

La varianza sirve para medir qué tan diferente es un gen entre pacientes. Si la varianza es muy baja, significa que es casi igual, mientras que si es alta significa que es muy diferente entre los pacientes. Necesitamos realizar un filtrado de genes de baja varianza para eliminarlos, debido a que si un gen tiene casi el mismo nivel de actividad en todos los pacientes, no dice nada sobre sus diferencias y por lo tanto no puede ayudar a separar grupos.

Para esto, utilizaremos una distribución 80/20, donde el 80% de los genes serán eliminados, después de ordenarlos de menor a mayor varianza, para conservar el 20% más variable y diferente entre pacientes. Se elige el 80% porque no queremos que sea muy agresivo como lo sería con un 95%, pero tampoco buscamos que sea muy conservador y haya mucho ruido debido a exceso de genes, como pasaría con una distribución del 50%. Después de aplicar el filtro, obtenemos que se eliminaron un total de 47,541 genes, quedando 11,886 con un umbral de varianza de 0.4712. Además, se generan dos gráficas:

- **Distribución de Varianza por Gen:** Muestra la varianza de todos los 59,427 genes antes del filtro. La gran mayoría se concentra cerca de cero, confirmando que la mayor parte del genoma tiene comportamiento casi constante entre pacientes y no aporta información útil para el clustering.
- **Genes Seleccionados** (varianza $\geq$ p80): Muestra únicamente los 11,886 genes conservados. Se puede observar que estos sí presentan varianza real y distribuida, lo que indica que su nivel de actividad cambia significativamente de paciente a paciente. Son estos genes los que contienen la información molecular necesaria para identificar subtipos del cáncer de colon.

```

In [13]: # PASO 3 – Filtrar genes de baja varianza
gene_var = expr_tumor.var(axis=1)
threshold = gene_var.quantile(0.80)
expr_filtered = expr_tumor[gene_var >= threshold]

print(f"Genes antes del filtro : {expr_tumor.shape[0]:,}")

```

```

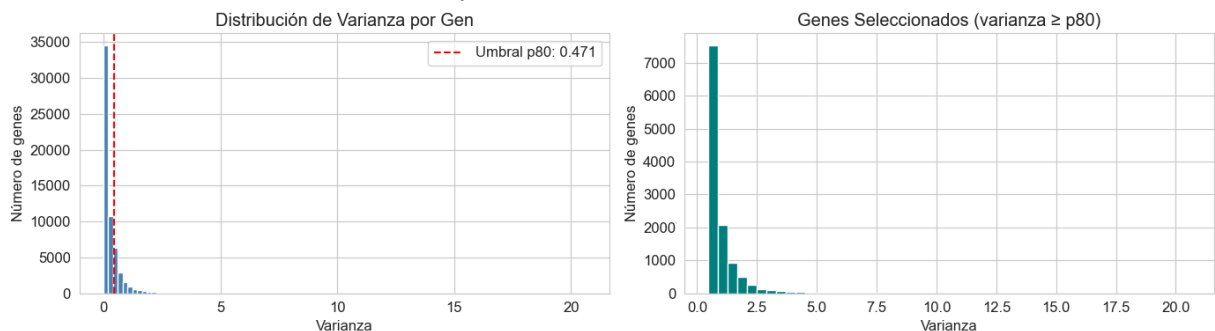
print(f"Umbral varianza (p80)   : {threshold:.4f}")
print(f"Genes después del filtro: {expr_filtered.shape[0]:,}")
print(f"Genes eliminados         : {expr_tumor.shape[0] - expr_filtered.shape[0]:,}")

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].hist(gene_var, bins=100, color='steelblue', edgecolor='white')
axes[0].axvline(threshold, color='red', linestyle='--',
                label=f'Umbral p80: {threshold:.3f}')
axes[0].set_title('Distribución de Varianza por Gen')
axes[0].set_xlabel('Varianza')
axes[0].set_ylabel('Número de genes')
axes[0].legend()

axes[1].hist(gene_var[gene_var >= threshold], bins=50,
            color='teal', edgecolor='white')
axes[1].set_title('Genes Seleccionados (varianza ≥ p80)')
axes[1].set_xlabel('Varianza')
axes[1].set_ylabel('Número de genes')
plt.tight_layout()
plt.savefig('fig_varianza_genes.png', dpi=150, bbox_inches='tight')
plt.show()

```

Genes antes del filtro : 59,427
 Umbral varianza (p80) : 0.4712
 Genes después del filtro: 11,886
 Genes eliminados : 47,541



Estandarización de datos

El último paso antes del clustering es la estandarización de los datos mediante z-score. Este paso es necesario porque los genes tienen niveles de expresión naturalmente distintos, algunos genes son muy activos y tienen valores altos por naturaleza, mientras otros son poco activos y tienen valores bajos. Sin estandarizar, el clustering estaría sesgado hacia los genes con valores numéricamente grandes, no por ser más informativos biológicamente sino simplemente por su escala.

La estandarización z-score transforma cada gen para que tenga media 0 y desviación estándar 1. Por ejemplo:

- **Gen ACTB** (gen que produce la **Betactina**): antes 12.0, 17.0 → después → -1.2, 1.8.
- **Gen TP53** (gen que produce la proteína **p53**): antes 0.1, 0.8 → después → -1.1, 1.5.

Aunque ACTB tenía valores 20 veces más grandes que TP53, después de estandarizar ambos están en la misma escala y contribuyen por igual al clustering. Como verificación, la matriz resultante muestra una media de -0.000000 y una desviación estándar de 1.000000, confirmando que la estandarización fue correcta. La matriz final lista para el clustering tiene 471 pacientes × 11,886 genes.

Para terminar, a diferencia del aprendizaje supervisado, en este proyecto no se realiza división en conjuntos de entrenamiento y prueba. Esto se debe a que el clustering es una técnica de aprendizaje no supervisado donde no existe una variable objetivo (Y) que predecir ni un resultado correcto contra el cual evaluar el modelo. El objetivo es descubrir estructura natural en los datos, por lo que se utilizan los 471 pacientes completos para el análisis. Dividir los datos reduciría innecesariamente la cantidad de información disponible para identificar patrones moleculares.

```
In [14]: # PASO 4 – Estandarización (z-score por gen)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
expr_scaled = scaler.fit_transform(expr_filtered.T)
expr_scaled_df = pd.DataFrame(expr_scaled,
                              index=expr_filtered.columns,
                              columns=expr_filtered.index)

print(f"Matriz final para clustering:")
print(f" Pacientes : {expr_scaled_df.shape[0]}")
print(f" Genes      : {expr_scaled_df.shape[1]:,}")
print(f" Media       : {expr_scaled_df.values.mean():.6f} (debe ser ~0)")
print(f" Desv. std    : {expr_scaled_df.values.std():.6f} (debe ser ~1)")
```

```
Matriz final para clustering:
Pacientes : 471
Genes      : 11,886
Media      : -0.000000 (debe ser ~0)
Desv. std  : 1.000000 (debe ser ~1)
```

Principal Component Analysis

El Análisis de Componentes Principales (PCA por sus siglas en inglés) es una técnica matemática que permite reducir la dimensionalidad de los datos, es decir, pasar de un espacio con muchas variables a uno con muchas menos, conservando la mayor cantidad de información posible. En nuestro caso, cada paciente tiene 11,886 genes como variables. Trabajar directamente con esas 11,886 dimensiones presenta dos problemas: primero, es computacionalmente muy costoso para el clustering; segundo, muchas de esas dimensiones contienen ruido o información redundante que puede dificultar encontrar patrones reales.

PCA resuelve esto encontrando nuevas variables llamadas componentes principales, que son combinaciones de los genes originales ordenadas de mayor a menor importancia. El

primer componente captura la mayor variación posible, el segundo la mayor variación restante, y así sucesivamente. De esta forma podemos quedarnos solo con los primeros componentes y descartar el resto sin perder mucha información relevante. En la siguiente celda se analizará cuántos componentes son necesarios para representar adecuadamente los datos, y se seleccionará el número óptimo basándose en las gráficas de varianza explicada.

```
In [15]: from sklearn.decomposition import PCA

# Calcular PCA con 100 componentes para ver mejor la curva
pca_full = PCA(n_components=100, random_state=42)
pca_full.fit(expr_scaled_df)

var_explicada = pca_full.explained_variance_ratio_
var_acumulada = np.cumsum(var_explicada)

# Scree plot + varianza acumulada
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

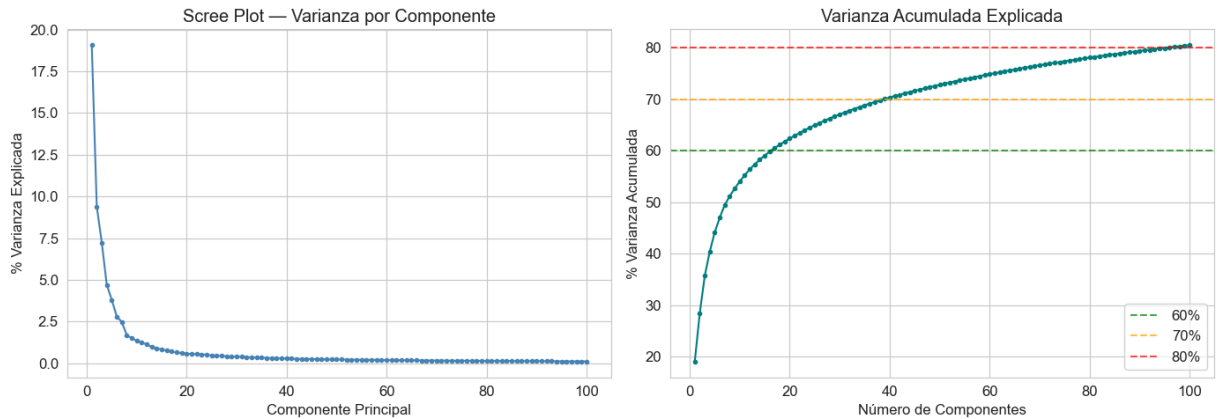
# Scree plot
axes[0].plot(range(1, 101), var_explicada * 100,
              'o-', color='steelblue', markersize=3)
axes[0].set_title('Scree Plot - Varianza por Componente')
axes[0].set_xlabel('Componente Principal')
axes[0].set_ylabel('% Varianza Explicada')

# Varianza acumulada
axes[1].plot(range(1, 101), var_acumulada * 100,
              'o-', color='teal', markersize=3)
axes[1].axhline(60, color='green', linestyle='--', alpha=0.7, label='60%')
axes[1].axhline(70, color='orange', linestyle='--', alpha=0.7, label='70%')
axes[1].axhline(80, color='red', linestyle='--', alpha=0.7, label='80%')
axes[1].set_title('Varianza Acumulada Explicada')
axes[1].set_xlabel('Número de Componentes')
axes[1].set_ylabel('% Varianza Acumulada')
axes[1].legend()

plt.tight_layout()
plt.savefig('fig_pca_varianza.png', dpi=150, bbox_inches='tight')
plt.show()

n_60 = np.argmax(var_acumulada >= 0.60) + 1
n_70 = np.argmax(var_acumulada >= 0.70) + 1
n_80 = np.argmax(var_acumulada >= 0.80) + 1

print(f"Varianza explicada PC1 : {var_explicada[0]*100:.2f}%")
print(f"Varianza explicada PC2 : {var_explicada[1]*100:.2f}%")
print(f"Varianza explicada PC3 : {var_explicada[2]*100:.2f}%")
print(f"Componentes para 60% : {n_60}")
print(f"Componentes para 70% : {n_70}")
print(f"Componentes para 80% : {n_80}")
print(f"Varianza con 50 comps : {var_acumulada[49]*100:.2f}%")
```



Varianza explicada PC1 : 19.07%
 Varianza explicada PC2 : 9.40%
 Varianza explicada PC3 : 7.24%
 Componentes para 60% : 17
 Componentes para 70% : 39
 Componentes para 80% : 96
 Varianza con 50 comps : 72.82%

```
In [16]: for i in range(10):
          print(f"PC{i+1}: {var_explicada[i]*100:.2f}%")
```

PC1: 19.07%
 PC2: 9.40%
 PC3: 7.24%
 PC4: 4.71%
 PC5: 3.80%
 PC6: 2.81%
 PC7: 2.48%
 PC8: 1.70%
 PC9: 1.50%
 PC10: 1.34%

Se aplicó PCA como paso de reducción de dimensionalidad donde primero se calcularon 100 componentes para analizar la varianza explicada mediante dos gráficas:

- **Scree Plot:** Muestra la varianza explicada por cada componente individual. Se observa una caída pronunciada en los primeros componentes: PC1 explica el 19.07%, PC2 el 9.40% y PC3 el 7.24%, seguida de una estabilización gradual. El codo de la curva se encuentra entre PC1 y PC2, donde se produce la caída más pronunciada de 19.07% a 9.40%, una reducción de casi 10 puntos. A partir de PC3 las caídas son graduales y menores a 3%, indicando rendimientos decrecientes.
- **Varianza Acumulada:** Muestra cuánta información total se retiene al aumentar el número de componentes. Se observa que se necesitan 17 componentes para alcanzar el 60% de varianza, 39 para el 70% y 96 para el 80%. Dado que usar 96 componentes en 471 pacientes no es práctico para clustering porque serían demasiadas variables y podrían fragmentar los grupos de tumores de forma artificial, y basándonos en la posición del codo del scree plot, se seleccionaron 17

componentes como número óptimo, reteniendo el 60.55% de la varianza total con la mayor eficiencia posible.

Con estas decisiones se logra un balance entre retener suficiente información biológica para identificar subgrupos reales, y mantener una dimensionalidad manejable para el clustering. Una vez seleccionados los 17 componentes, se procede a visualizar los pacientes en este espacio reducido para observar si existen patrones o separaciones visibles antes de aplicar el clustering.

```
In [17]: # Aplicar PCA con 17 componentes (60% varianza explicada, codo del scree plot)
N_COMPONENTS = 17

pca = PCA(n_components=N_COMPONENTS, random_state=42)
pca_coords = pca.fit_transform(expr_scaled_df)
pca_df = pd.DataFrame(pca_coords,
                      index=expr_scaled_df.index,
                      columns=[f'PC{i+1}' for i in range(N_COMPONENTS)])

print(f"Componentes seleccionados : {N_COMPONENTS}")
print(f"Varianza total explicada : {pca.explained_variance_ratio_.sum()*100:.1f}%")
print(f"Shape resultante : {pca_df.shape}")

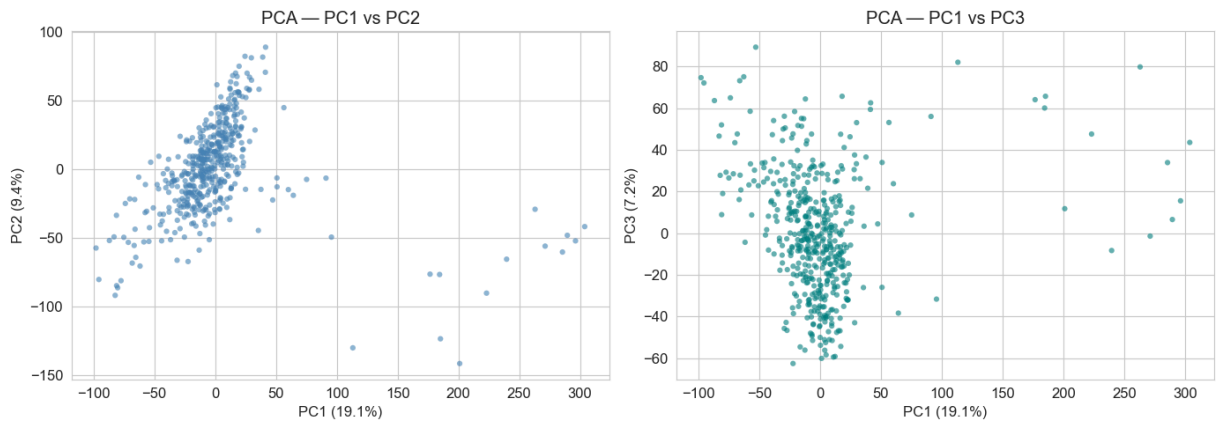
# PC1 vs PC2 y PC1 vs PC3
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(pca_df['PC1'], pca_df['PC2'],
               alpha=0.6, s=20, color='steelblue', edgecolors='none')
axes[0].set_title('PCA - PC1 vs PC2')
axes[0].set_xlabel(f'PC1 ({var_explicada[0]*100:.1f}%)')
axes[0].set_ylabel(f'PC2 ({var_explicada[1]*100:.1f}%)')

axes[1].scatter(pca_df['PC1'], pca_df['PC3'],
               alpha=0.6, s=20, color='teal', edgecolors='none')
axes[1].set_title('PCA - PC1 vs PC3')
axes[1].set_xlabel(f'PC1 ({var_explicada[0]*100:.1f}%)')
axes[1].set_ylabel(f'PC3 ({var_explicada[2]*100:.1f}%)')

plt.tight_layout()
plt.savefig('fig_pca_scatter.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
Componentes seleccionados : 17
Varianza total explicada : 60.55%
Shape resultante : (471, 17)
```



Con los 17 componentes seleccionados se visualizaron los 471 pacientes en el espacio PCA:

- **PC1 vs PC2:** La distribución muestra una nube diagonal orientada de abajo-izquierda a arriba-derecha, con una masa densa central y puntos más dispersos hacia los extremos. Se insinúan al menos dos zonas de densidad distinta, lo que sugiere la existencia de subgrupos moleculares en los datos. Los puntos con valores extremos en PC1 mayores a 150 corresponden probablemente a las 8 muestras TCGA-A6 identificadas como outliers anteriormente.
- **PC1 vs PC3:** La distribución es más compacta y centrada en el origen. PC3 aporta menos separación que PC2, con los puntos más concentrados verticalmente. Los mismos puntos extremos en PC1 aparecen nuevamente, confirmando que son muestras con un perfil molecular distinto al resto.

En general, las visualizaciones PCA sugieren que existe estructura en los datos aunque los subgrupos no son perfectamente separables visualmente lo cual es normal en datos de expresión génica reales donde las diferencias entre subtipos son graduales. El clustering en los siguientes pasos permitirá identificar estos grupos de forma más objetiva y precisa.

Clustering

El clustering es una técnica de aprendizaje no supervisado que busca agrupar automáticamente los pacientes en grupos, de tal forma que los pacientes dentro de un mismo grupo sean lo más similares posible entre sí, y lo más diferentes posible a los pacientes de otros grupos. A diferencia del aprendizaje supervisado, no se le dice al algoritmo cuáles son los grupos correctos, sino que los descubre por sí solo a partir de los patrones en los datos.

En este proyecto se aplicarán dos métodos de clustering sobre los 17 componentes principales obtenidos en la sección anterior. El primero es K-Means, que divide los pacientes en K grupos asignando cada uno al centroide más cercano de forma iterativa. El segundo es Hierarchical Clustering, que construye un árbol jerárquico uniendo

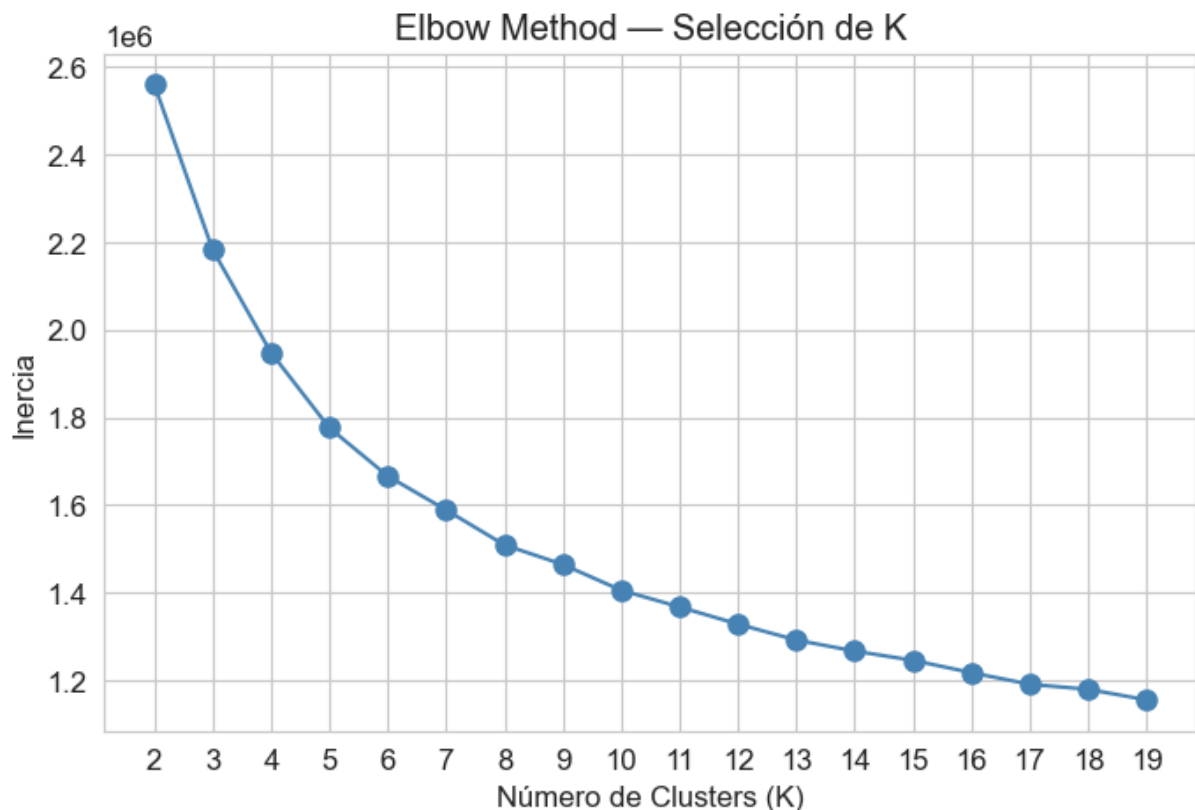
progresivamente los pacientes más similares hasta formar grupos. Ambos métodos serán comparados al final para evaluar qué tan consistentes son sus resultados. El método que produzca grupos más balanceados e interpretables será seleccionado para el análisis clínico y de genes de las secciones siguientes.

```
In [18]: from sklearn.cluster import KMeans

inertias = []
K_range = range(2, 20)

for k in K_range:
    kmeans = KMeans(n_clusters=k, n_init=20, random_state=42)
    kmeans.fit(pca_df)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(K_range, inertias, 'o-', color='steelblue', markersize=8)
plt.title('Elbow Method — Selección de K')
plt.xlabel('Número de Clusters (K)')
plt.ylabel('Inercia')
plt.xticks(K_range)
plt.savefig('fig_elbow.png', dpi=150, bbox_inches='tight')
plt.show()
```



Para determinar el número óptimo de clusters en K-Means se emplearon tres criterios complementarios: el **Elbow Method**, el **Silhouette Score** y el **Davies-Bouldin Index (DBI)**.

- **Elbow Method** consiste en graficar la inercia para distintos valores de K y localizar el punto de inflexión. En nuestra gráfica la curva es relativamente suave: de K=2 a K=3 se observa la caída más pronunciada, y es a partir de K=4 que la tendencia cambia de ritmo notablemente. En K=5 la caída ya es menos pronunciada y a partir de K=6 la curva tiende a estabilizarse.
- **Silhouette Score** mide qué tan bien está asignado cada punto a su cluster en comparación con el cluster más cercano (entre -1 y 1; mayor = mejor separación).
- **Davies-Bouldin Index** mide la relación entre la dispersión interna y la distancia entre clusters (menor = mejor). Ambos índices proveen una medida objetiva e independiente del criterio visual para seleccionar K.

Dado que el codo no es perfectamente claro, la decisión se apoya en dos criterios complementarios. Primero, visualmente K=4 representa el primer punto donde se observa un cambio visiblemente notable. Segundo, K=4 coincide con lo que la literatura científica describe para el cáncer de colon: cuatro subtipos moleculares conocidos (CMS1–CMS4) con comportamientos biológicos distintos. Que el método matemático y la biología apunten al mismo número refuerza que K=4 es una elección razonable.

```
In [19]: # K-Means con K=4
K_OPTIMO = 4

kmeans = KMeans(n_clusters=K_OPTIMO, n_init=50, random_state=42)
kmeans_labels = kmeans.fit_predict(pca_df)

# Agregar etiquetas
pca_df['Cluster_KMeans'] = kmeans_labels

print(f"K-Means con K={K_OPTIMO}, n_init=50")
print(f"\nDistribución de clusters:")
for k in range(K_OPTIMO):
    n = (kmeans_labels == k).sum()
    print(f" Cluster {k+1}: {n} pacientes ({n/len(kmeans_labels)*100:.1f}%)

# Visualizar clusters
colors = ['#e41a1c', '#377eb8', '#4daf4a', '#984ea3']
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for k in range(K_OPTIMO):
    mask = kmeans_labels == k
    axes[0].scatter(pca_df.loc[mask, 'PC1'], pca_df.loc[mask, 'PC2'],
                    c=colors[k], label=f'Cluster {k+1}',
                    alpha=0.7, s=25, edgecolors='none')
axes[0].set_title('K-Means - PC1 vs PC2')
axes[0].set_xlabel('PC1 (19.1%)')
axes[0].set_ylabel('PC2 (9.4%)')
axes[0].legend()

for k in range(K_OPTIMO):
    mask = kmeans_labels == k
```

```

axes[1].scatter(pca_df.loc[mask, 'PC1'], pca_df.loc[mask, 'PC3'],
                c=colors[k], label=f'Cluster {k+1}',
                alpha=0.7, s=25, edgecolors='none')
axes[1].set_title('K-Means — PC1 vs PC3')
axes[1].set_xlabel('PC1 (19.1%)')
axes[1].set_ylabel('PC3 (7.2%)')
axes[1].legend()

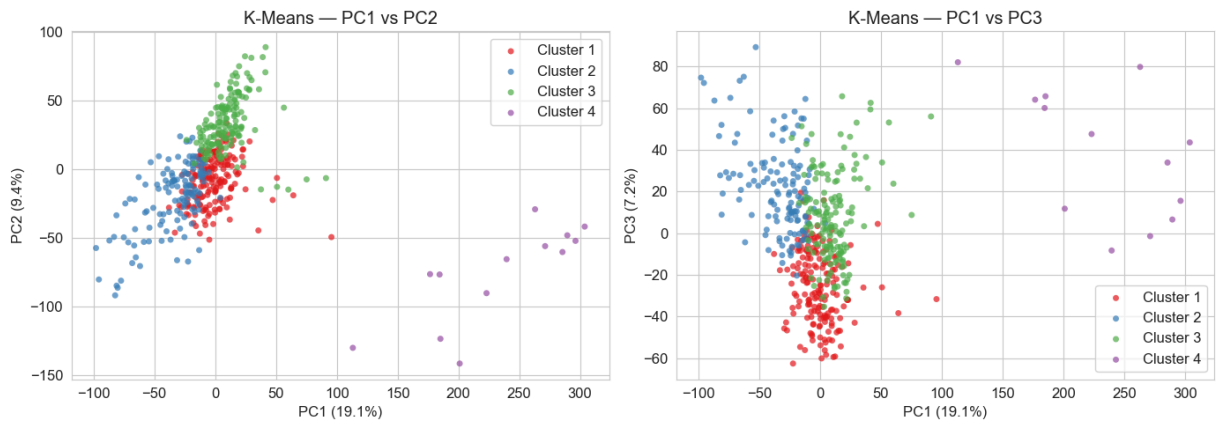
plt.tight_layout()
plt.savefig('fig_kmeans_clusters.png', dpi=150, bbox_inches='tight')
plt.show()

```

K-Means con K=4, n_init=50

Distribución de clusters:

Cluster 1: 176 pacientes (37.4%)
 Cluster 2: 131 pacientes (27.8%)
 Cluster 3: 151 pacientes (32.1%)
 Cluster 4: 13 pacientes (2.8%)



Se aplicó K-Means con K=4 y se corrió el algoritmo 50 veces con diferentes puntos de partida, seleccionando siempre la mejor solución. Esto reduce la posibilidad de obtener un resultado subóptimo por mala inicialización. Se fijó también una semilla aleatoria para garantizar que los resultados sean reproducibles.

La distribución de grupos resultante fue:

- Cluster 1: 176 pacientes (37.4%)
- Cluster 2: 131 pacientes (27.8%)
- Cluster 3: 151 pacientes (32.1%)
- Cluster 4: 13 pacientes (2.8%)

Los Clusters 1, 2 y 3 tienen tamaños balanceados y representan los tres subgrupos moleculares principales del análisis. El Cluster 4, con solo 13 pacientes, aparece claramente separado hacia los extremos del espacio PCA en ambas gráficas. Esto corresponde exactamente a lo que se anticipó en la sección de outliers: las muestras con perfil de expresión atípico (principalmente las TCGA-A6 con $z > 3$) fueron aisladas de forma automática por el algoritmo sin ninguna indicación explícita. Este resultado valida el correcto funcionamiento de K-Means y confirma que esas muestras son

genuinamente distintas al resto. El Cluster 4 se trata como un grupo técnico a lo largo de todo el análisis y sus resultados se interpretan con cautela. Los hallazgos moleculares y clínicos relevantes provienen de los Clusters 1, 2 y 3.

En las visualizaciones se observa que los Clusters 1, 2 y 3 se solapan parcialmente en el espacio PCA, lo cual es normal en datos de expresión génica reales donde los subtipos moleculares tienen transiciones graduales entre ellos.

Hierarchical Clustering

Se aplicó Hierarchical Clustering como segundo método de agrupamiento. A diferencia de K-Means, este algoritmo no necesita que se le diga de antemano cuántos grupos buscar. En cambio, construye un árbol jerárquico completo llamado dendrograma, que permite ver todas las posibles agrupaciones al mismo tiempo.

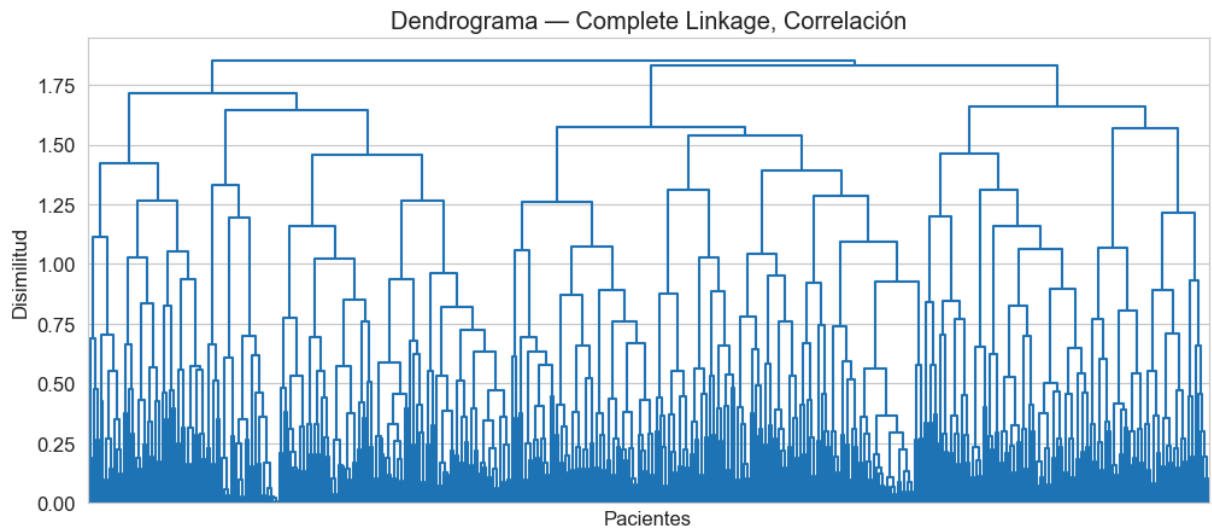
Para este método se utilizó distancia de correlación con enlace completo (complete linkage). La distancia de correlación mide la similitud entre los patrones de expresión de los pacientes independientemente de la magnitud de sus valores, lo cual es una forma natural de comparar perfiles genómicos. El enlace completo define la distancia entre dos grupos como la distancia máxima entre cualquier par de puntos de ambos grupos, produciendo clusters más compactos y bien definidos.

- **Dendrograma monocromático:** Muestra la estructura jerárquica completa de los pacientes. El eje vertical representa qué tan distintos son los grupos que se están uniendo en cada paso; entre más alto es el punto de unión, más diferentes son los grupos. Se observan grandes ramas principales en la parte superior del árbol con subdivisiones internas que sugieren subgrupos adicionales.
- **Dendrogramas policromáticos:** Se exploró el resultado con 2, 3 y 4 grupos para facilitar la comparación visual directa con K-Means.

```
In [20]: from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from scipy.spatial.distance import pdist

# Hierarchical clustering – complete linkage, correlación
dist_matrix = pdist(pca_df.drop('Cluster_KMeans', axis=1), metric='correlation')
Z_corr = linkage(dist_matrix, method='complete')

plt.figure(figsize=(12, 5))
dendrogram(Z_corr, no_labels=True, color_threshold=0)
plt.title('Dendrograma – Complete Linkage, Correlación')
plt.xlabel('Pacientes')
plt.ylabel('Disimilitud')
plt.savefig('fig_dendrograma_mono.png', dpi=150, bbox_inches='tight')
plt.show()
```



```
In [21]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

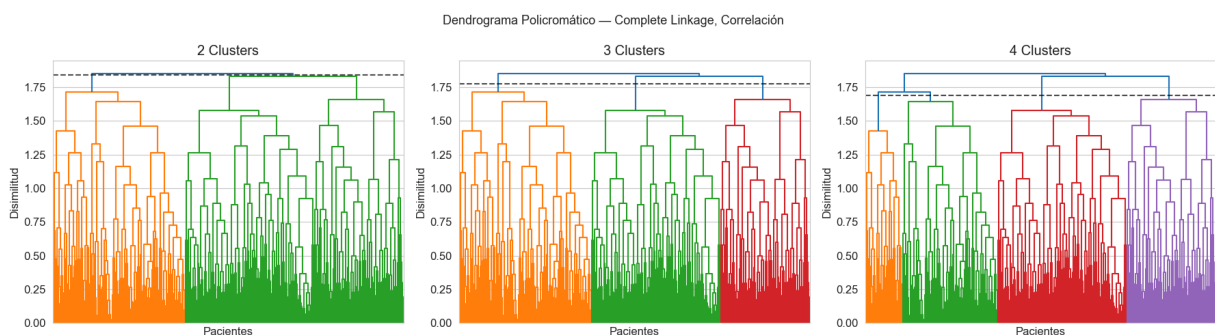
for idx, n_clusters in enumerate([2, 3, 4]):
    heights = sorted(Z_corr[:, 2], reverse=True)
    threshold = (heights[n_clusters-2] + heights[n_clusters-1]) / 2

    dendrogram(Z_corr, no_labels=True,
               color_threshold=threshold,
               ax=axes[idx])
    axes[idx].set_title(f'{n_clusters} Clusters')
    axes[idx].set_xlabel('Pacientes')
    axes[idx].set_ylabel('Disimilitud')
    axes[idx].axhline(y=threshold, color='black',
                     linestyle='--', alpha=0.7)

plt.suptitle('Dendrograma Policromático — Complete Linkage, Correlación', fc
plt.tight_layout()
plt.savefig('fig_dendrograma_poli.png', dpi=150, bbox_inches='tight')
plt.show()

# Clusters finales con correlación
hier_labels = fcluster(Z_corr, 4, criterion='maxclust')
pca_df['Cluster_Hier'] = hier_labels

print("Distribución Hierarchical Clustering (4 clusters):")
for k in range(1, 5):
    n = (hier_labels == k).sum()
    print(f" Cluster {k}: {n} pacientes ({n/len(hier_labels)*100:.1f}%)")
```



Distribución Hierarchical Clustering (4 clusters):

- Cluster 1: 50 pacientes (10.6%)
- Cluster 2: 128 pacientes (27.2%)
- Cluster 3: 173 pacientes (36.7%)
- Cluster 4: 120 pacientes (25.5%)

Comparación K-Means y Hierarchical Clustering

Para evaluar qué tan parecidos son los resultados de ambos métodos se calculó el Adjusted Rand Index (ARI), que mide la similitud entre dos formas de agrupar los mismos datos. Un valor de 0 indica que la coincidencia es la misma que se esperaría por azar, y un valor de 1 indica que ambos métodos asignaron exactamente los mismos grupos.

Es importante notar que K-Means utiliza distancia euclidiana mientras que Hierarchical utiliza distancia de correlación. Esto significa que cada método mide la similitud entre pacientes de forma distinta, lo que puede explicar parte de las diferencias en los resultados. A pesar de ello, la comparación sigue siendo válida porque el objetivo no es verificar que los métodos sean idénticos, sino evaluar si encuentran estructura similar en los datos independientemente del criterio utilizado.

La tabla de contingencia muestra cuántos pacientes de cada grupo de K-Means terminaron en cada grupo de Hierarchical. Los grupos con alta correspondencia entre métodos representan subgrupos moleculares sólidos, que pueden ser encontrados independientemente del algoritmo utilizado. Los grupos con asignación más dispersa representan zonas donde la separación entre pacientes es más gradual y ambos algoritmos la interpretan de forma algo distinta.

Se selecciona K-Means como modelo principal para el análisis de genes y el análisis clínico de las secciones siguientes, por producir grupos balanceados y reproducibles de forma eficiente.

```
In [22]: # Comparación K-Means vs Hierarchical
from sklearn.metrics import adjusted_rand_score

# ARI entre los dos métodos
ari = adjusted_rand_score(kmeans_labels, hier_labels)
print(f"Adjusted Rand Index (similitud entre métodos): {ari:.4f}")
print(f"(0 = aleatorio, 1 = idénticos)")

# Tabla de contingencia
contingencia = pd.crosstab(
    pd.Series(kmeans_labels, name='K-Means'),
    pd.Series(hier_labels, name='Hierarchical')
)
print(f"\nTabla de contingencia:")
print(contingencia)

# Visualización comparativa
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors_k = ['#e41a1c', '#377eb8', '#4daf4a', '#984ea3']
```

```

colors_h = ['#ff7f00', '#33a02c', '#e31a1c', '#6a3d9a']

for k in range(4):
    mask = kmeans_labels == k
    axes[0].scatter(pca_df.loc[mask, 'PC1'], pca_df.loc[mask, 'PC2'],
                    c=colors_k[k], label=f'Cluster {k+1}',
                    alpha=0.7, s=25, edgecolors='none')
axes[0].set_title('K-Means (K=4)')
axes[0].set_xlabel('PC1 (19.1%)')
axes[0].set_ylabel('PC2 (9.4%)')
axes[0].legend()

for k in range(1, 5):
    mask = hier_labels == k
    axes[1].scatter(pca_df.loc[mask, 'PC1'], pca_df.loc[mask, 'PC2'],
                    c=colors_h[k-1], label=f'Cluster {k}',
                    alpha=0.7, s=25, edgecolors='none')
axes[1].set_title('Hierarchical (K=4, Correlación)')
axes[1].set_xlabel('PC1 (19.1%)')
axes[1].set_ylabel('PC2 (9.4%)')
axes[1].legend()

plt.suptitle('Comparación K-Means vs Hierarchical Clustering', fontsize=13)
plt.tight_layout()
plt.savefig('fig_comparacion_clustering.png', dpi=150, bbox_inches='tight')
plt.show()

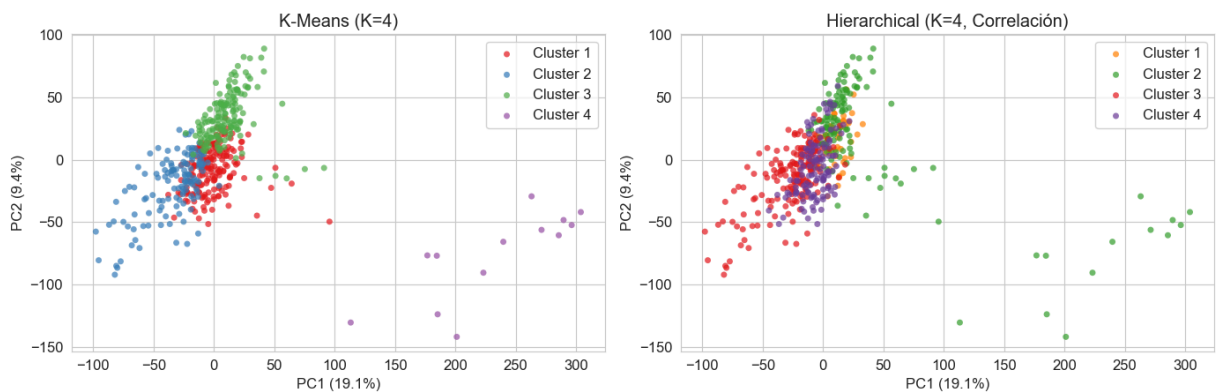
```

Adjusted Rand Index (similitud entre métodos): 0.3219
(0 = aleatorio, 1 = idénticos)

Tabla de contingencia:

Hierarchical	1	2	3	4
K-Means				
0	24	32	29	91
1	0	0	127	4
2	26	83	17	25
3	0	13	0	0

Comparación K-Means vs Hierarchical Clustering



Análisis de Genes

Una vez identificados los clusters, el siguiente paso es entender qué genes son los responsables de las diferencias entre ellos. Esto es importante porque los clusters fueron contruidos únicamente con patrones matemáticos en los datos, sin saber qué genes los estaban causando. Al identificar qué genes tienen expresión significativamente distinta entre grupos, podemos darle un significado biológico real a cada cluster.

Para esto se realizarán dos análisis. Primero, una prueba estadística sobre todos los genes para identificar cuáles tienen diferencias significativas entre los 4 clusters. Segundo, una visualización mediante heatmap de los genes más relevantes, para observar visualmente los patrones de expresión que caracterizan a cada grupo. Es importante recordar que en esta sección se trabaja exclusivamente con los datos de expresión génica y los clusters obtenidos, sin incorporar aún ninguna variable clínica.

Análisis Estadístico de Genes Diferenciales

Para identificar los genes que más contribuyen a diferenciar los grupos obtenidos, se aplicó la prueba estadística de Kruskal-Wallis sobre los genes del dataset filtrado. Esta prueba evalúa si el nivel de actividad de cada gen es significativamente distinto entre los grupos sin necesidad de asumir que los datos siguen una distribución normal, lo cual es lo más adecuado para datos de expresión génica.

Como se analizaron miles de genes al mismo tiempo, existe la posibilidad de encontrar resultados significativos simplemente por azar. Para controlar esto se aplicó la corrección de Bonferroni, que ajusta cada p-value multiplicándolo por el número total de pruebas realizadas.

La proporción de genes significativos tras la corrección es alta, lo cual es esperable y debe entenderse en contexto: antes de esta prueba ya se habían eliminado los genes con poca variación entre pacientes (filtrado por varianza). Es decir, los genes que quedaron son precisamente los que más cambian de paciente a paciente, por lo que es natural que también muestren diferencias entre grupos. Esto no invalida los resultados; simplemente significa que la tasa de significancia refleja ese subconjunto más informativo y no al genoma completo.

Los 10 genes con mayor significancia estadística incluyen:

- **CEP170:** controla cómo se divide una célula en dos. Si está mal regulado, las células se dividen de forma descontrolada, que es exactamente lo que hace un tumor.
- **ZEB2:** le da a las células cancerosas la capacidad de moverse y escapar del tumor original hacia otras partes del cuerpo.
- **ANKRD12:** funciona como un interruptor que decide qué otros genes se encienden o apagan dentro de la célula.

- **MIR5739**: molécula pequeña que regula otros genes. Su papel exacto en cáncer de colon todavía se está estudiando.
- **POLR2I**: parte de la maquinaria que la célula usa para leer su propio ADN y fabricar proteínas. Pertenece a la misma familia que POLR2J, un gen mencionado en el artículo TCGA 2012 como marcador asociado a agresividad tumoral.
- **CSGALNACT2**: produce el material que rodea y sostiene a las células. Los tumores alteran este material para expandirse más fácilmente.
- **FAU**: participa en la síntesis de proteínas y en el sistema que decide qué proteínas se eliminan dentro de la célula. Cuando este sistema falla, se acumulan proteínas dañinas.
- **NDUFS7** y **NDUFA11**: controlan cómo las células producen su energía. Los tumores modifican este proceso para tener más combustible y crecer más rápido.
- **HEG1**: promueve la formación de nuevos vasos sanguíneos. Los tumores lo usan para construir su propio sistema de nutrición y oxigenación.

```
In [23]: from scipy.stats import kruskal
import warnings
warnings.filterwarnings('ignore')

expr_analysis = expr_filtered.T.copy()
expr_analysis.index = expr_scaled_df.index
expr_analysis['Cluster'] = kmeans_labels

print("Calculando Kruskal-Wallis para todos los genes...")

grupos_por_cluster = {k: expr_analysis[expr_analysis['Cluster'] == k].drop('
                        for k in range(4))

pvalues = {}
for i, gene in enumerate(expr_filtered.index):
    grupos = [grupos_por_cluster[k][gene].values for k in range(4)]
    _, p = kruskal(*grupos)
    pvalues[gene] = p
    if i % 1000 == 0:
        print(f" Procesados: {i:,} / {len(expr_filtered.index):,}")

pval_series = pd.Series(pvalues)
pval_corr = (pval_series * len(pval_series)).clip(upper=1.0)
sig_genes = pval_corr[pval_corr < 0.05]

print(f"\nTotal genes analizados      : {len(pval_series):,}")
print(f"Genes significativos (p<0.05) : {len(sig_genes):,}")
print(f"\nTop 10 genes más diferenciales:")
print(pval_corr.nsmallest(10))
```


Calculando Kruskal-Wallis para todos los genes...

```
Procesados: 0 / 11,886
Procesados: 1,000 / 11,886
Procesados: 2,000 / 11,886
Procesados: 3,000 / 11,886
Procesados: 4,000 / 11,886
Procesados: 5,000 / 11,886
Procesados: 6,000 / 11,886
Procesados: 7,000 / 11,886
Procesados: 8,000 / 11,886
Procesados: 9,000 / 11,886
Procesados: 10,000 / 11,886
Procesados: 11,000 / 11,886
```

Total genes analizados : 11,886

Genes significativos ($p < 0.05$) : 9,451

Top 10 genes más diferenciales:

```
CEP170      8.610150e-57
ZEB2        1.223498e-56
MIR5739     6.760916e-56
POLR2I      4.152180e-53
ANKRD12     9.474064e-53
NDUFA11     1.968079e-52
HEG1        2.163969e-51
CSGALNACT2  3.274530e-51
FAUP1       5.717052e-51
NDUFS7      1.074020e-50
```

dtype: float64

Heatmap de Genes Más Significativos

Un heatmap es una representación visual donde los colores indican niveles de actividad génica. El color rojo significa que el gen está muy activo en ese paciente, el azul que está poco activo o apagado, y el blanco que tiene un nivel intermedio. Cada columna representa un paciente ordenado por grupo y cada fila representa un gen, con los valores ajustados para que todos los genes sean comparables en la misma escala.

Se visualizaron los 40 genes más significativos estadísticamente. Los pacientes están ordenados por grupo, identificados con una barra de color en la parte superior:

- Cluster 1 → Rojo
- Cluster 2 → Azul
- Cluster 3 → Verde
- Cluster 4 → Morado

El heatmap revela tres patrones principales de comportamiento entre grupos:

- **Grupo 1** — Alta actividad en Cluster 2: Genes como **NDUFA11**, **NDUFS7** y **POLR2I** muestran color rojo intenso en el Cluster 2 y azul en los demás. Estos genes controlan cómo las células producen energía y cómo fabrican proteínas. Esto

sugiere que el Cluster 2 tiene una maquinaria celular más activa en términos de metabolismo interno.

- **Grupo 2** — Baja actividad en todos los grupos: Los genes **MIR5739**, **MIR5706** y **MIR605** aparecen sin color en todos los grupos. Son moléculas reguladoras pequeñas con poca actividad en todos los pacientes sin importar el grupo al que pertenezcan.
- **Grupo 3** — Alta actividad en Clusters 1 y 3: Genes como **ZEB2**, **WIPF1**, **HEG1**, **CEP170** y **ANKRD12** muestran el patrón opuesto al Grupo 1, rojos en Clusters 1 y 3 y azules en Cluster 2. ZEB2 es conocido por darle a las células cancerosas la capacidad de moverse e invadir otros tejidos. WIPF1 está relacionado con los cambios internos que necesita la célula para poder desplazarse. Esto sugiere que los Clusters 1 y 3 tienen un perfil molecular más agresivo en términos de invasión.
- **Cluster 4**: aparece como una franja muy pequeña en el heatmap y no muestra un patrón claro propio, lo cual es consistente con lo documentado anteriormente: estos pacientes corresponden a muestras con perfiles atípicos de probable origen técnico.

En resumen: el **Cluster 2** agrupa tumores con alta actividad metabólica pero menor potencial invasivo. Los **Clusters 1 y 3** agrupan tumores con alta expresión de genes relacionados con invasión y migración celular, lo que los hace molecularmente más peligrosos desde esa perspectiva.

```
In [24]: import seaborn as sns

# Usar los genes más significativos del Kruskal-Wallis
top_genes = pval_corr.nsmallest(40).index.tolist()
print(f"Top 40 genes más significativos:")
print(top_genes)

# Ordenar pacientes por cluster
label_series = pd.Series(kmeans_labels, index=expr_scaled_df.index)
order = label_series.sort_values()

# Seleccionar solo columnas que existen en expr_filtered
valid_samples = [s for s in order.index if s in expr_filtered.columns]
valid_labels = label_series[valid_samples].sort_values()

heatmap_data = expr_filtered.loc[top_genes, valid_labels.index]

# Estandarizar
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
heatmap_scaled = pd.DataFrame(
    scaler.fit_transform(heatmap_data.T).T,
    index=heatmap_data.index,
    columns=heatmap_data.columns
)

# Colores por cluster
```

```

color_map = {0: '#e41a1c', 1: '#377eb8', 2: '#4daf4a', 3: '#984ea3'}
col_colors = valid_labels.map(color_map)

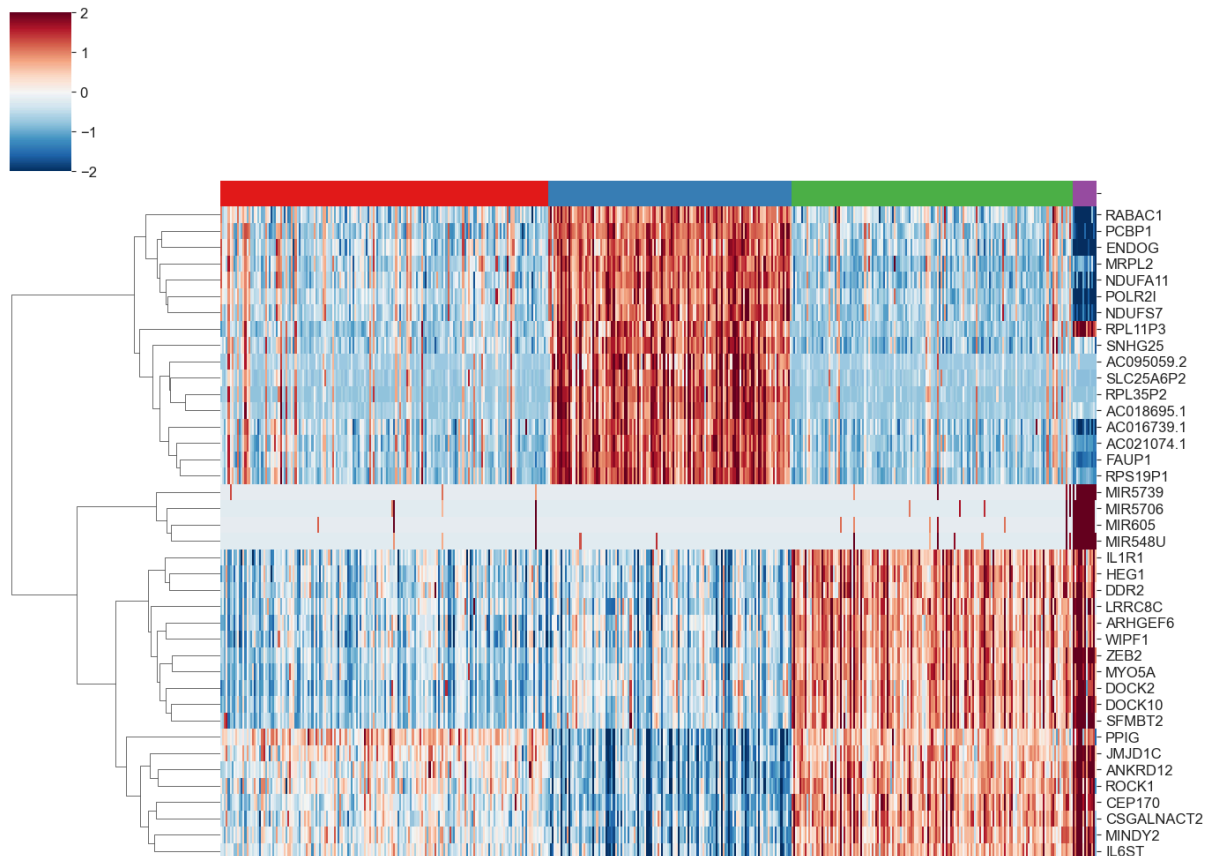
# Heatmap
g = sns.clustermap(heatmap_scaled,
                    col_colors=col_colors,
                    row_cluster=True,
                    col_cluster=False,
                    cmap='RdBu_r',
                    center=0,
                    vmin=-2, vmax=2,
                    xticklabels=False,
                    yticklabels=True,
                    figsize=(14, 10))
g.fig.suptitle('Heatmap - Top 40 Genes Más Significativos por Cluster',
               fontsize=13, y=1.02)
plt.savefig('fig_heatmap_genes.png', dpi=150, bbox_inches='tight')
plt.show()

```

Top 40 genes más significativos:

['CEP170', 'ZEB2', 'MIR5739', 'POLR2I', 'ANKRD12', 'NDUFA11', 'HEG1', 'CSGALNACT2', 'FAUP1', 'NDUFS7', 'MIR605', 'MIR5706', 'MIR548U', 'DOCK10', 'JMJD1C', 'IL1R1', 'MYO5A', 'SNHG25', 'SLC25A6P2', 'ARHGEF6', 'RPL35P2', 'ROCK1', 'MINDY2', 'AC016739.1', 'AC021074.1', 'RPS19P1', 'PCBP1', 'AC018695.1', 'DDR2', 'LRRC8C', 'DOCK2', 'AC095059.2', 'RPL11P3', 'RABAC1', 'PPIG', 'WIPF1', 'IL6ST', 'SFMBT2', 'MRPL2', 'ENDOG']

Heatmap — Top 40 Genes Más Significativos por Cluster



Análisis clínico

Una vez identificados los grupos mediante K-Means, se incorporan los datos clínicos para analizar si los grupos moleculares obtenidos presentan diferencias en variables clínicas relevantes. Este análisis permite determinar si los subtipos moleculares también tienen implicaciones reales para la salud de los pacientes.

Para variables categóricas como el estadio tumoral y el sexo se utilizó la prueba **Chi-cuadrado**, que evalúa si la distribución de categorías es independiente del grupo. Para variables numéricas como la edad se utilizó **Kruskal-Wallis**, la misma prueba utilizada en el análisis de genes. Para la supervivencia se utilizaron **curvas de Kaplan-Meier** con un Log-rank test para comparar las curvas entre grupos.

```
In [25]: # Unir clusters con datos clínicos
cluster_df = pd.DataFrame({
    'sample': expr_scaled_df.index,
    'Cluster': kmeans_labels + 1
})

# Unir con clinical
clin_cluster = clin.merge(cluster_df, on='sample', how='inner')
print(f"Pacientes con datos clínicos y cluster: {len(clin_cluster)}")
print(f"\nDistribución por cluster:")
print(clin_cluster['Cluster'].value_counts().sort_index())
```

Pacientes con datos clínicos y cluster: 471

Distribución por cluster:

Cluster

1 176

2 131

3 151

4 13

Name: count, dtype: int64

Stage Tumoral y Sexo por Cluster

Stage tumoral: La prueba Chi-cuadrado arrojó un p-value de 0.4906, lo que indica que no existe una diferencia estadísticamente significativa en la distribución de estadios entre los clusters. Sin embargo, visualmente se observa que el Cluster 3 tiene una mayor proporción de Stage II y una menor proporción de Stage IV comparado con los demás grupos, lo que podría sugerir una tendencia hacia estadios menos avanzados en ese cluster.

Sexo: La prueba Chi-cuadrado arrojó un p-value de 0.4437, indicando que la distribución de género es prácticamente igual entre los 4 clusters. Esto confirma que los clusters no están sesgados por sexo y que las diferencias moleculares encontradas son independientes del género del paciente.

```
In [26]: from scipy.stats import chi2_contingency

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
```

```

# Stage simplificado
def simplify_stage(s):
    if pd.isna(s): return 'Unknown'
    if 'IV' in s: return 'Stage IV'
    if 'III' in s: return 'Stage III'
    if 'II' in s: return 'Stage II'
    if 'I' in s: return 'Stage I'
    return 'Unknown'

clin_cluster['Stage_simple'] = clin_cluster['ajcc_pathologic_stage.diagnoses']

# Gráfica Stage por cluster
stage_counts = pd.crosstab(clin_cluster['Cluster'],
                           clin_cluster['Stage_simple'])
stage_counts = stage_counts[['Stage I', 'Stage II', 'Stage III', 'Stage IV']]
stage_pct = stage_counts.div(stage_counts.sum(axis=1), axis=0) * 100

stage_pct.plot(kind='bar', ax=axes[0], colormap='RdYlGn_r', edgecolor='white')
axes[0].set_title('Distribución de Stage Tumoral por Cluster')
axes[0].set_xlabel('Cluster')
axes[0].set_ylabel('% de pacientes')
axes[0].legend(title='Stage', bbox_to_anchor=(1.05, 1))
axes[0].tick_params(axis='x', rotation=0)

# Chi-cuadrado Stage
stage_known = clin_cluster[clin_cluster['Stage_simple'] != 'Unknown']
ct_stage = pd.crosstab(stage_known['Cluster'], stage_known['Stage_simple'])
chi2, p_stage, dof, _ = chi2_contingency(ct_stage)
axes[0].set_title(f'Stage Tumoral por Cluster\n(Chi² p={p_stage:.4f})')

# Gráfica Sexo por cluster
gender_counts = pd.crosstab(clin_cluster['Cluster'],
                             clin_cluster['gender.demographic'])
gender_pct = gender_counts.div(gender_counts.sum(axis=1), axis=0) * 100

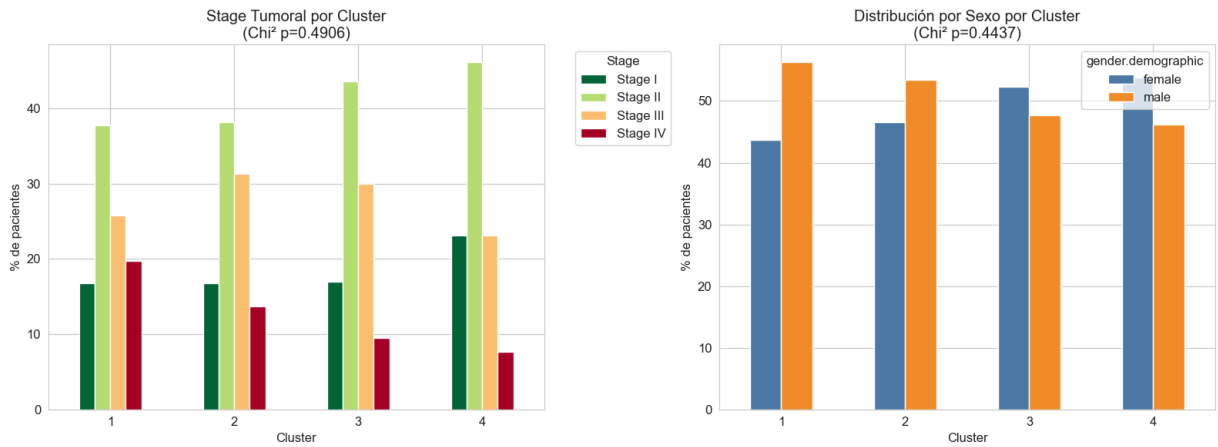
gender_pct.plot(kind='bar', ax=axes[1], color=['#4e79a7', '#f28e2b'],
                edgecolor='white')
axes[1].set_title('Distribución por Sexo por Cluster')
axes[1].set_xlabel('Cluster')
axes[1].set_ylabel('% de pacientes')
axes[1].tick_params(axis='x', rotation=0)

ct_gender = pd.crosstab(clin_cluster['Cluster'],
                        clin_cluster['gender.demographic'])
chi2_g, p_gender, _, _ = chi2_contingency(ct_gender)
axes[1].set_title(f'Distribución por Sexo por Cluster\n(Chi² p={p_gender:.4f})')

plt.tight_layout()
plt.savefig('fig_clinico_categorico.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"Chi² Stage - p-value: {p_stage:.6f}")
print(f"Chi² Género - p-value: {p_gender:.6f}")

```



Chi² Stage – p-value: 0.490596

Chi² Género – p-value: 0.443659

Edad por Cluster

La distribución de edad al diagnóstico muestra patrones similares entre los clusters. La prueba Kruskal-Wallis arrojó un p-value de 0.0896, que aunque no alcanza el umbral de significancia estadística convencional de 0.05, se encuentra en el límite y podría sugerir una tendencia leve entre grupos. Analizando cada cluster individualmente:

- Cluster 1: Media de 66.6 años y mediana de 68.0 años. Es el grupo más numeroso con 176 pacientes y presenta una distribución de edad amplia, con pacientes desde los 30 hasta los 90 años aproximadamente. Representa el rango de edad más típico del cáncer de colon.
- Cluster 2: Media de 69.0 años y mediana de 70.0 años. Es el cluster con la edad más avanzada entre los tres grupos principales. Su mayor edad promedio podría estar relacionada con el perfil molecular menos agresivo que mostró en el análisis de genes, ya que tumores de menor agresividad suelen manifestarse en pacientes de mayor edad.
- Cluster 3: Media de 65.5 años y mediana de 68.0 años. Es el cluster con la menor edad media entre los grupos principales. Esto es notable porque en el análisis de genes este cluster mostró un perfil más agresivo con alta expresión de genes de invasión como ZEB2. Tumores más agresivos tienden a aparecer a edades más tempranas, lo cual es consistente con este resultado.
- Cluster 4: Media de 71.9 años y mediana de 74.0 años. Es el cluster con la mayor edad promedio de todos, aunque con solo 13 pacientes este resultado debe interpretarse con cautela ya que no es representativo estadísticamente.

En conjunto, aunque las diferencias no son estadísticamente significativas, la tendencia observada es biológicamente coherente con los perfiles moleculares identificados en la sección anterior.

```
In [27]: from scipy.stats import kruskal

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```

# Boxplot edad por cluster
colors = ['#e41a1c', '#377eb8', '#4daf4a', '#984ea3']
data_edad = [clin_cluster[clin_cluster['Cluster'] == k]['age_at_index.demogr
               for k in range(1, 5)]

bp = axes[0].boxplot(data_edad, patch_artist=True, labels=['C1', 'C2', 'C3',
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
axes[0].set_title('Distribución de Edad por Cluster')
axes[0].set_xlabel('Cluster')
axes[0].set_ylabel('Edad (años)')

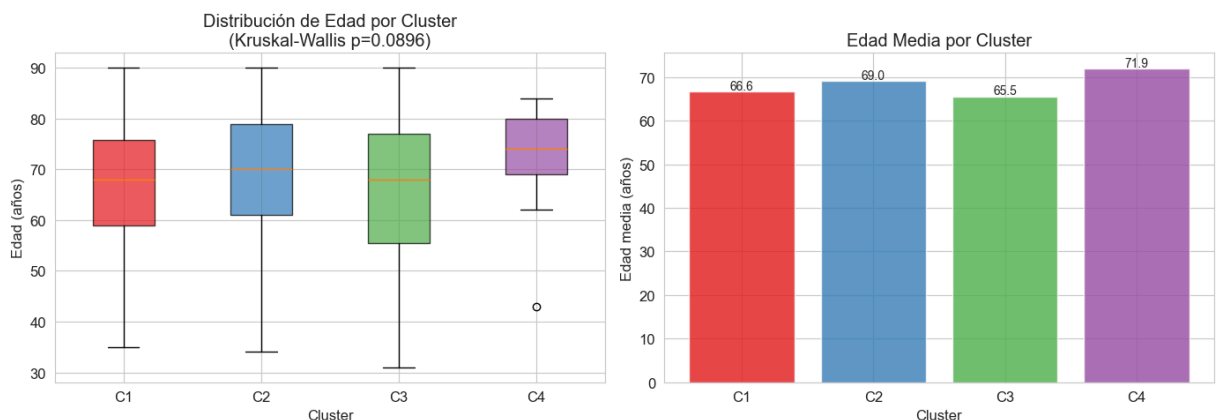
# Kruskal-Wallis edad
stat, p_edad = kruskal(*data_edad)
axes[0].set_title(f'Distribución de Edad por Cluster\n(Kruskal-Wallis p={p_e

# Medias de edad
medias_edad = [clin_cluster[clin_cluster['Cluster'] == k]['age_at_index.demogr
               for k in range(1, 5)]
axes[1].bar(['C1', 'C2', 'C3', 'C4'], medias_edad,
            color=colors, edgecolor='white', alpha=0.8)
axes[1].set_title('Edad Media por Cluster')
axes[1].set_xlabel('Cluster')
axes[1].set_ylabel('Edad media (años)')
for i, v in enumerate(medias_edad):
    axes[1].text(i, v + 0.5, f'{v:.1f}', ha='center', fontsize=10)

plt.tight_layout()
plt.savefig('fig_clinico_edad.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"Kruskal-Wallis Edad - p-value: {p_edad:.6f}")
for k in range(1, 5):
    media = clin_cluster[clin_cluster['Cluster'] == k]['age_at_index.demogr
    mediana = clin_cluster[clin_cluster['Cluster'] == k]['age_at_index.demogr
    print(f" Cluster {k}: media={media:.1f}, mediana={mediana:.1f}")

```



Kruskal-Wallis Edad – p-value: 0.089613

Cluster 1: media=66.6, mediana=68.0

Cluster 2: media=69.0, mediana=70.0

Cluster 3: media=65.5, mediana=68.0

Cluster 4: media=71.9, mediana=74.0

Curvas de Supervivencia por Cluster

Las curvas de Kaplan-Meier muestran la probabilidad de supervivencia a lo largo del tiempo para cada cluster. El Log-rank test arrojó un p-value de 0.6663, lo que indica que las diferencias entre las curvas no son estadísticamente significativas en términos globales. Sin embargo, visualmente se observan diferencias clínicamente interesantes. El Cluster 2 muestra la mejor supervivencia a largo plazo, estabilizándose alrededor del 63% tras los 2,000 días. El Cluster 3 muestra el peor pronóstico, con una caída pronunciada que llega al 27% en los seguimientos más largos. Los Clusters 1 y 3 muestran trayectorias similares en los primeros 1,500 días pero divergen posteriormente. El Cluster 4 tiene muy pocos eventos registrados debido a su pequeño tamaño, por lo que su curva no es representativa.

La falta de significancia estadística puede deberse al tamaño de muestra y al tiempo de seguimiento disponible. Las tendencias visuales observadas son consistentes con la interpretación molecular de la sección anterior, donde el Cluster 2 mostraba un perfil menos agresivo y el Cluster 3 uno más invasivo.

```
In [28]: # Instalar lifelines si no está
import subprocess
subprocess.run(['pip', 'install', 'lifelines', '--break-system-packages', '-

from lifelines import KaplanMeierFitter
from lifelines.statistics import multivariate_logrank_test

# Unir survival con clusters
surv_cluster = surv.merge(cluster_df, on='sample', how='inner')
print(f"Pacientes con datos de supervivencia y cluster: {len(surv_cluster)}")

# Kaplan-Meier por cluster
fig, ax = plt.subplots(figsize=(10, 6))
colors = ['#e41a1c', '#377eb8', '#4daf4a', '#984ea3']

for k in range(1, 5):
    mask = surv_cluster['Cluster'] == k
    kmf = KaplanMeierFitter()
    kmf.fit(surv_cluster.loc[mask, 'OS.time'],
            surv_cluster.loc[mask, 'OS'],
            label=f'Cluster {k} (n={mask.sum()})')
    kmf.plot_survival_function(ax=ax, color=colors[k-1], ci_show=False)

# Log-rank test
results = multivariate_logrank_test(
    surv_cluster['OS.time'],
    surv_cluster['Cluster'],
```



```

surv_cluster['OS']
)

ax.set_title(f'Curvas de Supervivencia por Cluster\n(Log-rank p={results.p_v
ax.set_xlabel('Días')
ax.set_ylabel('Probabilidad de supervivencia')
ax.legend()
plt.tight_layout()
plt.savefig('fig_supervivencia.png', dpi=150, bbox_inches='tight')
plt.show()

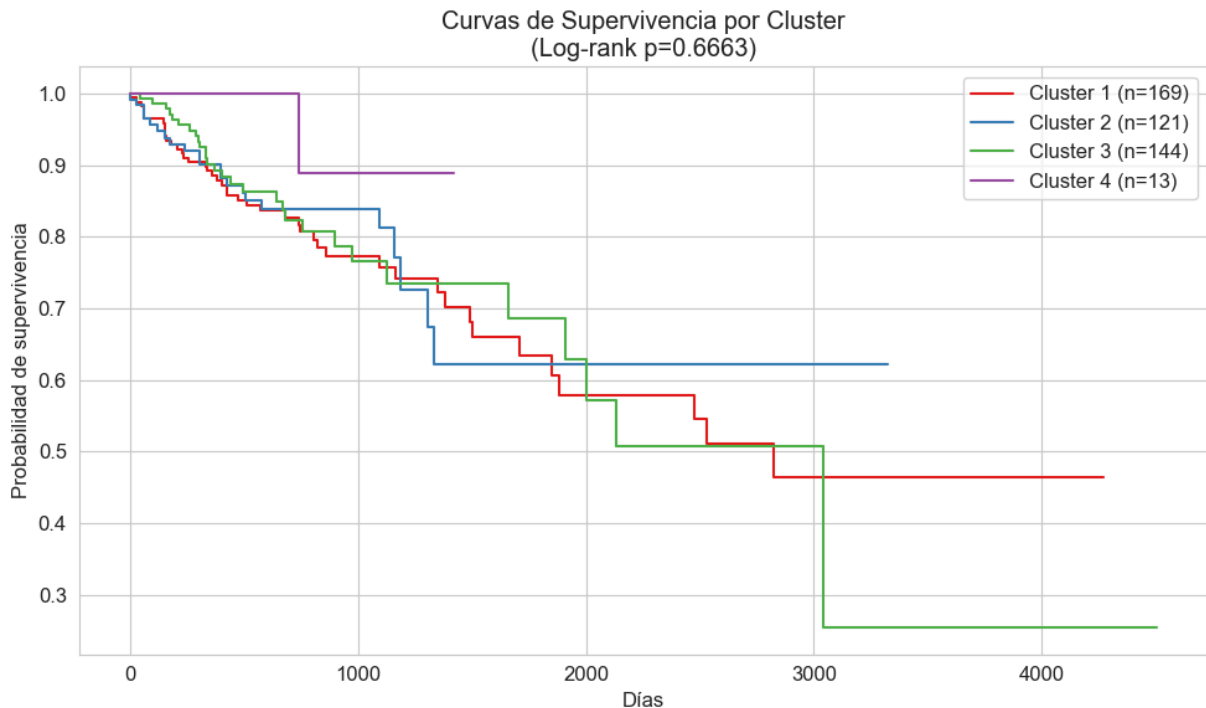
print(f"Log-rank test p-value: {results.p_value:.6f}")

```

[notice] A new release of pip is available: 26.0.1 -> 26.1.1

[notice] To update, run: `pip install --upgrade pip`

Pacientes con datos de supervivencia y cluster: 447



Log-rank test p-value: 0.666326

Podemos concluir que cuando comparamos los grupos con la información médica de los pacientes, las diferencias no fueron suficientemente grandes como para ser estadísticamente confiables. Es decir, no podemos afirmar con certeza que los grupos tienen distinto stage, distinta edad o distinto sexo.

Sin embargo, al ver las curvas de supervivencia sí notamos algo interesante pues los pacientes del Cluster 2 vivieron más tiempo en promedio y los pacientes del Cluster 3 tuvieron el peor pronóstico. Esto tiene mucho sentido con lo que encontramos en el análisis de genes: los tumores con más capacidad de invadir otros tejidos (Cluster 3) resultaron ser también los que tuvieron peor supervivencia.

Discusión de Artículos

Contexto del artículo de referencia

El artículo seleccionado para esta discusión es "Comprehensive Molecular Characterization of Human Colon and Rectal Cancer", publicado en la revista *Nature* en 2012 por The Cancer Genome Atlas Network. Este estudio es especialmente relevante porque utiliza exactamente los mismos datos que este proyecto, ya que los datos disponibles en UCSC Xena provienen directamente del TCGA.

El estudio analizó 276 muestras de cáncer colorrectal utilizando varios tipos de información al mismo tiempo: secuenciación del genoma, número de copias de ADN, patrones de metilación y niveles de expresión de ARN mensajero y microARN. Su objetivo principal era identificar alteraciones genómicas y caracterizar molecularmente el cáncer colorrectal de forma integral.

Entre sus hallazgos más importantes, el artículo identificó que el 16% de los tumores son hipermutados, con tres cuartas partes de estos mostrando alta inestabilidad de microsatélites (MSI-alto). También encontró que la vía de señalización **WNT** estaba alterada en el 93% de todos los tumores, y que genes como **APC**, **TP53**, **KRAS** y **PIK3CA** son los más frecuentemente mutados. Mediante agrupamiento no supervisado sobre perfiles de expresión de ARN mensajero, el artículo separó los tumores en **tres grupos distintos**, uno de los cuales coincidía en gran medida con los tumores hipermutados CIMP-alto.

Similitudes con el proyecto

Ambos estudios usan los mismos datos (TCGA) y ambos aplican técnicas de agrupamiento no supervisado sobre datos de expresión génica para identificar subgrupos moleculares en el cáncer de colon. En ambos casos se encontró evidencia de que los tumores no son todos iguales y pueden dividirse en grupos con características moleculares distintas.

Existe además una conexión directa entre los resultados de este proyecto y los del artículo. **POLR2I**, que aparece en nuestro top de genes más significativos, pertenece a la familia ARN Polimerasa II. Su gen relacionado **POLR2J** aparece explícitamente en la Figura 5b del artículo como marcador de expresión asociado a agresividad tumoral, junto con otros genes como **SCN5A** y **APOL6**. Esto sugiere que la señal molecular que detectamos en **POLR2I** tiene relevancia biológica real y es consistente con lo que el estudio TCGA encontró mediante un análisis más completo.

HEG1, también altamente diferencial en nuestro análisis, está relacionado con la formación de nuevos vasos sanguíneos, un proceso que el artículo describe como marcador de comportamiento tumoral agresivo. Del mismo modo, **ZEB2**, uno de nuestros genes más diferenciales, está relacionado con la capacidad de las células cancerosas de moverse e invadir tejidos cercanos, proceso central en la progresión tumoral descrita en el artículo.

Diferencias con este proyecto

El artículo utilizó varios tipos de datos al mismo tiempo, incluyendo mutaciones, metilación y número de copias del ADN, mientras que este proyecto se limitó exclusivamente a los datos de expresión génica. Esto hace que la caracterización del artículo sea más completa, aunque también considerablemente más compleja de realizar.

En cuanto al número de subgrupos, el artículo identificó tres grupos de expresión de ARNm, mientras que nuestro análisis identificó cuatro. Sin embargo, excluyendo el Cluster 4 de probable origen técnico, la convergencia hacia tres subgrupos biológicos reales es coherente con los resultados del artículo.

El artículo señala que no contaba con datos de supervivencia para la mayoría de sus muestras, ya que provenían de una colección reciente. En cambio, este análisis incluyó curvas de Kaplan-Meier que, aunque sin significancia estadística, mostraron tendencias clínicas interesantes entre los grupos.

Por último, el artículo realizó el agrupamiento directamente sobre los perfiles completos de expresión sin reducción de dimensionalidad previa. Nuestro enfoque de aplicar PCA antes del agrupamiento busca reducir el ruido en los datos, lo que puede producir grupos más estables al trabajar con tantas variables.

Conclusiones

Este análisis demostró que los tumores de cáncer de colon analizados no son todos iguales. Aunque todos los pacientes tienen el mismo tipo de cáncer, a nivel molecular sus tumores se comportan de maneras distintas. Usando únicamente los datos de actividad génica y sin saber de antemano cuántos grupos existían, los algoritmos identificaron subgrupos de pacientes con perfiles moleculares distintos que son sólidos y detectables por más de un método de agrupamiento.

Los Clusters 1, 2 y 3 representan subtipos moleculares reales con características propias. El Cluster 2 agrupa tumores con alta actividad en genes de producción de energía pero menor capacidad de invasión, mientras que los Clusters 1 y 3 muestran alta expresión de genes relacionados con movimiento e invasión celular (ZEB2, WIPF1, HEG1). Esta diferencia tiene sentido biológico y es coherente con los tres grupos de expresión descritos en el artículo TCGA 2012 con los mismos datos. El Cluster 4, con solo 13 pacientes, corresponde a muestras con perfiles atípicos de probable origen técnico y debe interpretarse con cautela.

Las curvas de supervivencia mostraron tendencias clínicamente interesantes. El cluster con perfil metabólico activo mostró mejor supervivencia a largo plazo que el cluster con perfil invasivo, aunque estas diferencias no alcanzaron significancia estadística, posiblemente por el tamaño de la muestra y el tiempo de seguimiento disponible.

Limitaciones del estudio:

La principal limitación es que solo se utilizó la actividad génica como fuente de información, dejando fuera datos importantes como las mutaciones del ADN, los patrones de metilación o el número de copias de genes. Incluir esa información, como hace el artículo TCGA 2012, daría una imagen más completa de cada subtipo.

Una segunda limitación importante es la ausencia del estado MSI/MSS en el análisis clínico. El artículo TCGA 2012 identifica a los tumores hipermutados con inestabilidad de microsatélites como un subgrupo con mejor pronóstico y diferente respuesta al tratamiento. Conectar los grupos obtenidos con esta variable sería el siguiente paso natural y podría explicar parte de las diferencias moleculares observadas.

Aunque las diferencias moleculares entre grupos son claras, estas no se tradujeron en diferencias clínicas estadísticamente significativas en estadio tumoral, sexo ni edad, lo que puede deberse al tamaño de la muestra o al tiempo de seguimiento disponible en el dataset.

Trabajo futuro:

Sería valioso combinar los datos de expresión génica con mutaciones, metilación y número de copias para obtener una caracterización más completa de cada subtipo. También sería interesante comparar formalmente los grupos encontrados con los cuatro Consensus Molecular Subtypes (CMS1-CMS4) del cáncer de colon, e incorporar el estado MSI como variable clínica clave. Finalmente, un mayor tiempo de seguimiento permitiría evaluar con mayor poder estadístico las diferencias de supervivencia entre los grupos.

Referencias

- Claude (Anthropic). (2026). Asistencia en el desarrollo de código [Herramienta de inteligencia artificial]. <https://claude.ai>
- Davidson-Pilon, C. (2019). lifelines: Survival analysis in Python. *Journal of Open Source Software*, 4(40), 1317. <https://doi.org/10.21105/joss.01317>
- Goldman, M. J., Craft, B., Hastie, M., Repčeka, K., McDade, F., Kamath, A., Banerjee, A., Luo, Y., Rogers, D., Brooks, A. N., Zhu, J., & Haussler, D. (2020). Visualizing and interpreting cancer genomics data via the Xena platform. *Nature Biotechnology*, 38(6), 675–678. <https://doi.org/10.1038/s41587-020-0546-8>
- Guinney, J., Dienstmann, R., Wang, X., de Reyniès, A., Schlicker, A., Soneson, C., Marisa, L., Roepman, P., Nyamundanda, G., Angelino, P., Bot, B. M., Morris, J. S., Simon, I. M., Milburn, M., Esteva, E., Cospedes, I., Eide, P. W., Pique, J. M., Rohr, S., Aschele, C., ... Sadanandam, A. (2015). The consensus molecular subtypes of

colorectal cancer. *Nature Medicine*, 21(11), 1350–1356.

<https://doi.org/10.1038/nm.3967>

- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Martínez Torteya, A. (2026). SC3314 *Inteligencia Artificial — Unidad 3: Aprendizaje No Supervisado* [Material de clase]. Universidad de Monterrey.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51–56. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- The Cancer Genome Atlas Network. (2012). Comprehensive molecular characterization of human colon and rectal cancer. *Nature*, 487(7407), 330–337. <https://doi.org/10.1038/nature11252>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., ... SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-020-0772-5>
- Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>